

Diario Proyecto Eclipse

2026-3-02 12:21

Status:

Labels: [Physics](#)

Empecé construyendo el hamiltoniano. Con una cosa que he asumido:

1. La posición del Sol es cte en el tiempo. (corrijo, no hace falta suponer esto, tomamos el baricentro del sistema solar, que es la posición del CM del sistema solar.)

Al empezar a plantear el hamiltoniano de la Tierra, he pensado hacerlo en esféricas. Pero luego, cuando estaba a punto de derivar r , me he dado cuenta de que las órbitas no son de $R=\text{cte}$ porque son elipses, entonces igual en esféricas es un jaleo. He decidido preguntarle a Gemini si existen coordenadas para elipses en R^3 , me ha dicho que sí, que existen: coordenadas elipsoidales confocales y esferoidales (prolatas y oblatas). Pero que no servían, porque estos sistemas se construyen sobre focos fijos, cosa que nuestro sistema no tiene. Me ha recomendado hacerlo en cartesianas, por que las esféricas tienen un par de problemas: 2. Complejidad del potencial 3. Singularidades en $\theta = 0, \pi$

Entonces, haciéndolo con cartesianas, $\vec{r}_T$ ha quedado:

$$\vec{r}_T = (x, y, z)$$

$$\dot{\vec{r}}_T = (\dot{x}, \dot{y}, \dot{z})$$

$$\dot{\vec{r}}_T^2 = \dot{x}^2 + \dot{y}^2 + \dot{z}^2$$

$$T = \frac{1}{2} m \dot{\vec{r}}_T^2 + \frac{1}{2} I \omega^2$$

Aquí, había pensado en hacer $v = w/r$, pero claro, eso es una condición de rodadura sin deslizamiento. Que no es el caso de un planeta. Así que lo dejamos como w .

Para V :

$$V = -G \frac{Mm}{\sqrt{x^2 + y^2 + z^2}}$$

Como r no depende explícitamente del tiempo, y V no depende de la velocidad, $H = T + V$.

$$H = \frac{1}{2}m(\dot{x}^2 + \dot{y}^2 + \dot{z}^2) + \frac{1}{5}mR^2w^2 - G\frac{Mm}{\sqrt{x^2 + y^2 + z^2}}$$

Ahora, obtengamos los momentos conjugados:

$$p_x = \frac{\partial L}{\partial \dot{x}}, \quad p_y = \frac{\partial L}{\partial \dot{y}}, \quad p_z = \frac{\partial L}{\partial \dot{z}}$$

Hallamos L :

$$L = T - V = \frac{1}{2}m(\dot{x}^2 + \dot{y}^2 + \dot{z}^2) + \frac{1}{5}mR^2w^2 + G\frac{Mm}{\sqrt{x^2 + y^2 + z^2}}$$

$$p_x = m\dot{x}, \quad p_y = m\dot{y}, \quad p_z = m\dot{z}$$

Luego, no podemos dejar w así, necesitamos ponerlo en función de coordenadas generalizadas:

$$w^2 = w_1^2 + w_2^2 + w_3^2$$

Usamos los ángulos de Euler: $\dot{\phi}$ (precesión); $\dot{\theta}$ (nutación); $\dot{\psi}$ (rotación propia)

$$\omega_1 = \dot{\phi} \sin \theta \sin \psi + \dot{\theta} \cos \psi$$

$$\omega_2 = \dot{\phi} \sin \theta \cos \psi - \dot{\theta} \sin \psi$$

$$\omega_3 = \dot{\phi} \cos \theta + \dot{\psi}$$

$$\omega_1^2 = \dot{\phi}^2 \sin^2 \theta \sin^2 \psi + \dot{\theta}^2 \cos^2 \psi + 2\dot{\phi}\dot{\theta} \sin \theta \sin \psi \cos \psi$$

$$\omega_2^2 = \dot{\phi}^2 \sin^2 \theta \cos^2 \psi + \dot{\theta}^2 \sin^2 \psi - 2\dot{\phi}\dot{\theta} \sin \theta \sin \psi \cos \psi$$

$$\omega_1^2 + \omega_2^2 = \dot{\phi}^2 \sin^2 \theta (\sin^2 \psi + \cos^2 \psi) + \dot{\theta}^2 (\cos^2 \psi + \sin^2 \psi) = \dot{\phi}^2 \sin^2 \theta + \dot{\theta}^2$$

$$|\vec{\omega}|^2 = \dot{\theta}^2 + \dot{\phi}^2 \sin^2 \theta + (\dot{\psi} + \dot{\phi} \cos \theta)^2$$

$$T_{rot} = \frac{1}{5}mR^2 \left[\dot{\theta}^2 + \dot{\phi}^2 \sin^2 \theta + (\dot{\psi} + \dot{\phi} \cos \theta)^2 \right]$$

Ahora sustituimos de nuevo en el L :

$$L = \frac{1}{2}m(\dot{x}^2 + \dot{y}^2 + \dot{z}^2) + \frac{1}{5}mR^2 \left[\dot{\theta}^2 + \dot{\phi}^2 \sin^2 \theta + (\dot{\psi} + \dot{\phi} \cos \theta)^2 \right] + G\frac{Mm}{\sqrt{x^2 + y^2 + z^2}}$$

Calculamos los momentos conjugados restantes:

$$p_\theta = \frac{2}{5}mR^2\dot{\theta}, \quad p_\psi = \frac{2}{5}mR^2(\dot{\psi} + \dot{\phi} \cos \theta), \quad p_\phi = \frac{2}{5}mR^2\dot{\phi} \sin^2 \theta + p_\psi \cos \theta$$

Ahora, sustituimos en H :

$$H = \frac{p_x^2 + p_y^2 + p_z^2}{2m} + \frac{5}{4mR^2} \left[p_\theta^2 + \frac{(p_\phi - p_\psi \cos \theta)^2}{\sin^2 \theta} + p_\psi^2 \right] - G \frac{Mm}{\sqrt{x^2 + y^2 + z^2}}$$

Me acabo de dar cuenta de una inconsistencia: hemos introducido la T_{rot} para ser lo más precisos posibles y no tener en cuenta a la Tierra como una esfera perfecta, pero el I que hemos usado si es el de una esfera perfecta, por lo que es un error a corregir.

Entonces, primero, calculamos el momento de inercia:

$$I_{ij} = \int_V \rho(\vec{r})(r^2 \delta_{ij} - x_i x_j) dV$$

$$\mathbf{I} = \left(\int \rho(y^2 + z^2) dV \quad - \int \rho xy dV \quad - \int \rho xz dV \quad - \int \rho xy dV \quad \int \rho(x^2 + z^2) dV \quad - \int \rho yz dV \quad - \int \rho xz dV \quad - \int \rho yz dV \quad \int \rho(x^2 + y^2) dV \right)$$

Tenemos que $I_1 = I_2$, y que $I_3 > I_1$. A $I_1 = I_2$ les vamos a llamar $I_\perp$, y a $I_3 = I_z$.

$$T_{rot} = \frac{1}{2} I_1 \omega_1^2 + \frac{1}{2} I_2 \omega_2^2 + \frac{1}{2} I_3 \omega_3^2 = \frac{1}{2} I_\perp (\omega_1^2 + \omega_2^2) + \frac{1}{2} I_z \omega_3^2$$

$$T_{rot} = \frac{1}{2} I_\perp (\dot{\theta}^2 + \dot{\phi}^2 \sin^2 \theta) + \frac{1}{2} I_z (\dot{\psi} + \dot{\phi} \cos \theta)^2$$

$$p_\theta = I_\perp \dot{\theta}, \quad p_\psi = I_z (\dot{\psi} + \dot{\phi} \cos \theta), \quad p_\phi = I_\perp \dot{\phi} \sin^2 \theta + I_z (\dot{\psi} + \dot{\phi} \cos \theta) \cos \theta$$

$$\dot{\theta} = \frac{p_\theta}{I_\perp}, \quad \dot{\phi} = \frac{p_\phi - p_\psi \cos \theta}{I_\perp \sin^2 \theta}, \quad \dot{\psi} + \dot{\phi} \cos \theta = \frac{p_\psi}{I_z}$$

$$H = \frac{p_x^2 + p_y^2 + p_z^2}{2m} + \frac{p_\theta^2}{2I_\perp} + \frac{(p_\phi - p_\psi \cos \theta)^2}{2I_\perp \sin^2 \theta} + \frac{p_\psi^2}{2I_z} + V(x, y, z, \phi, \theta)$$

Lo mismo, en este caso, Gemini ha puntualizado correctamente que no puedo suponer que la Tierra no es esférica para T pero sí para V . Entonces, vamos a usar el desarrollo multipolar (como el que se usa en electricidad) para el V .

Para ello, usaremos esta expresión obtenida del Goldstein:

$$V = -\frac{GMm}{r} - \frac{GM(I_z - I_\perp)}{2r^3} (3 \cos^2 \alpha - 1)$$

Pero, tenemos que desglosarlo para ponerlo en las fórmulas que queremos:

$$r = \sqrt{x^2 + y^2 + z^2}$$

$$\hat{u}_p = (\sin \theta \sin \phi, -\sin \theta \cos \phi, \cos \theta)$$

$$\cos \alpha = \frac{x \sin \theta \sin \phi - y \sin \theta \cos \phi + z \cos \theta}{\sqrt{x^2 + y^2 + z^2}}$$

$$V(x, y, z, \phi, \theta) = -\frac{GMm}{\sqrt{x^2 + y^2 + z^2}} - \frac{GM(I_z - I_\perp)}{2(x^2 + y^2 + z^2)^{3/2}} \left[3 \left(\frac{x \sin \theta \sin \phi - y \sin \theta \cos \phi + z \cos \theta}{\sqrt{x^2 + y^2 + z^2}} \right)^2 - 1 \right]$$

$$H = \frac{p_x^2 + p_y^2 + p_z^2}{2m} + \frac{p_\theta^2}{2I_\perp} + \frac{(p_\phi - p_\psi \cos \theta)^2}{2I_\perp \sin^2 \theta} + \frac{p_\psi^2}{2I_z} - \frac{GMm}{\sqrt{x^2 + y^2 + z^2}} - \frac{GM(I_z - I_\perp)}{2(x^2 + y^2 + z^2)^{3/2}} \left[3 \left(\frac{x \sin \theta}{\sqrt{x^2 + y^2 + z^2}} \right) \right]$$

En este H hay un problema, solo tiene en cuenta un cuerpo que genera perturbaciones, que es el que se denota con la masa M . Pero en nuestro sistema, vamos a usar la Luna y el Sol. Por lo que tenemos que tener en cuenta ambos potenciales, por lo que el H que usaremos es este:

$$H = T_{tras} + H_{rot,\oplus} + V_{SL} + V_{\oplus L} + V_{\oplus S}$$

$$H = \frac{p_{x_S}^2 + p_{y_S}^2 + p_{z_S}^2}{2m_S} + \frac{p_{x_\oplus}^2 + p_{y_\oplus}^2 + p_{z_\oplus}^2}{2m_\oplus} + \frac{p_{x_L}^2 + p_{y_L}^2 + p_{z_L}^2}{2m_L} + \frac{p_\theta^2}{2I_\perp} + \frac{(p_\phi - p_\psi \cos \theta)^2}{2I_\perp \sin^2 \theta} + \frac{p_\psi^2}{2I_z}$$

El círculo con cruz representa a la Tierra.

Una vez tenemos H , tenemos que hallar las ecuaciones canónicas: $\dot{q}_k = \frac{\partial H}{\partial p_k}$ y $\dot{p}_k = -\frac{\partial H}{\partial q_k}$

Ecuaciones de la cinemática

Traslación del Sol:

$$\dot{x}_S = \frac{p_{x_S}}{m_S}, \quad \dot{y}_S = \frac{p_{y_S}}{m_S}, \quad \dot{z}_S = \frac{p_{z_S}}{m_S}$$

Traslación de la Tierra:

$$\dot{x}_\oplus = \frac{p_{x_\oplus}}{m_\oplus}, \quad \dot{y}_\oplus = \frac{p_{y_\oplus}}{m_\oplus}, \quad \dot{z}_\oplus = \frac{p_{z_\oplus}}{m_\oplus}$$

Traslación de la Luna:

$$\dot{x}_L = \frac{p_{x_L}}{m_L}, \quad \dot{y}_L = \frac{p_{y_L}}{m_L}, \quad \dot{z}_L = \frac{p_{z_L}}{m_L}$$

Rotación de la Tierra:

$$\dot{\theta} = \frac{p_\theta}{I_\perp}, \quad \dot{\phi} = \frac{p_\phi - p_\psi \cos \theta}{I_\perp \sin^2 \theta}, \quad \dot{\psi} = \frac{p_\psi}{I_z} - \frac{(p_\phi - p_\psi \cos \theta) \cos \theta}{I_\perp \sin^2 \theta}$$

Ecuaciones de la dinámica

Dinámica del Sol:

$$\dot{p}_{x_S} = -\frac{\partial V}{\partial x_S}, \quad \dot{p}_{y_S} = -\frac{\partial V}{\partial y_S}, \quad \dot{p}_{z_S} = -\frac{\partial V}{\partial z_S}$$

Dinámica de la Tierra:

$$\dot{p}_{x_{\oplus}} = -\frac{\partial V}{\partial x_{\oplus}}, \quad \dot{p}_{y_{\oplus}} = -\frac{\partial V}{\partial y_{\oplus}}, \quad \dot{p}_{z_{\oplus}} = -\frac{\partial V}{\partial z_{\oplus}}$$

Dinámica de la Luna:

$$\dot{p}_{x_L} = -\frac{\partial V}{\partial x_L}, \quad \dot{p}_{y_L} = -\frac{\partial V}{\partial y_L}, \quad \dot{p}_{z_L} = -\frac{\partial V}{\partial z_L}$$

Dinámica de la rotación de la Tierra:

$$\dot{p}_{\theta} = \frac{(p_{\phi} - p_{\psi} \cos \theta)^2 \cos \theta}{I_{\perp} \sin^3 \theta} - \frac{(p_{\phi} - p_{\psi} \cos \theta)p_{\psi}}{I_{\perp} \sin \theta} - \frac{\partial V}{\partial \theta}$$
$$\dot{p}_{\phi} = -\frac{\partial V}{\partial \phi}, \quad \dot{p}_{\psi} = 0$$

Para luego poder meterlo en Python, vamos a crear variables auxiliares que hagan más sencillo la fase de programación:

- $x_{SL} = x_L - x_S, \quad y_{SL} = y_L - y_S, \quad z_{SL} = z_L - z_S \quad \Rightarrow \quad r_{SL} = \sqrt{x_{SL}^2 + y_{SL}^2 + z_{SL}^2}$
- $x_{\oplus L} = x_L - x_{\oplus}, \quad y_{\oplus L} = y_L - y_{\oplus}, \quad z_{\oplus L} = z_L - z_{\oplus} \quad \Rightarrow \quad r_{\oplus L} = \sqrt{x_{\oplus L}^2 + y_{\oplus L}^2 + z_{\oplus L}^2}$
- $x_{\oplus S} = x_S - x_{\oplus}, \quad y_{\oplus S} = y_S - y_{\oplus}, \quad z_{\oplus S} = z_S - z_{\oplus} \quad \Rightarrow \quad r_{\oplus S} = \sqrt{x_{\oplus S}^2 + y_{\oplus S}^2 + z_{\oplus S}^2}$
- $u_L = x_{\oplus L} \sin \theta \sin \phi - y_{\oplus L} \sin \theta \cos \phi + z_{\oplus L} \cos \theta$
- $u_S = x_{\oplus S} \sin \theta \sin \phi - y_{\oplus S} \sin \theta \cos \phi + z_{\oplus S} \cos \theta$
- $u_{\theta L} = \frac{\partial u_L}{\partial \theta} = x_{\oplus L} \cos \theta \sin \phi - y_{\oplus L} \cos \theta \cos \phi - z_{\oplus L} \sin \theta$
- $u_{\phi L} = \frac{\partial u_L}{\partial \phi} = x_{\oplus L} \sin \theta \cos \phi + y_{\oplus L} \sin \theta \sin \phi$
- $u_{\theta S} = \frac{\partial u_S}{\partial \theta} = x_{\oplus S} \cos \theta \sin \phi - y_{\oplus S} \cos \theta \cos \phi - z_{\oplus S} \sin \theta$
- $u_{\phi S} = \frac{\partial u_S}{\partial \phi} = x_{\oplus S} \sin \theta \cos \phi + y_{\oplus S} \sin \theta \sin \phi$
- $C_L = \frac{3Gm_L(I_z - I_{\perp})}{2r_{\oplus L}^5}, \quad C_S = \frac{3Gm_S(I_z - I_{\perp})}{2r_{\oplus S}^5}$

Y ahora tenemos todas las ecuaciones:

1. $\dot{x}_S = \frac{p_{x_S}}{m_S}$
2. $\dot{y}_S = \frac{p_{y_S}}{m_S}$
3. $\dot{z}_S = \frac{p_{z_S}}{m_S}$
4. $\dot{x}_{\oplus} = \frac{p_{x_{\oplus}}}{m_{\oplus}}$

5. $\dot{y}_\oplus = \frac{p_{y_\oplus}}{m_\oplus}$
 6. $\dot{z}_\oplus = \frac{p_{z_\oplus}}{m_\oplus}$
 7. $\dot{x}_L = \frac{p_{x_L}}{m_L}$
 8. $\dot{y}_L = \frac{p_{y_L}}{m_L}$
 9. $\dot{z}_L = \frac{p_{z_L}}{m_L}$
 10. $\dot{\theta} = \frac{p_\theta}{I_\perp}$
 11. $\dot{\phi} = \frac{p_\phi - p_\psi \cos \theta}{I_\perp \sin^2 \theta}$
 12. $\dot{\psi} = \frac{p_\psi}{I_z} - \frac{(p_\phi - p_\psi \cos \theta) \cos \theta}{I_\perp \sin^2 \theta}$
 13. $\dot{p}_{x_S} = \frac{Gm_S m_L}{r_{SL}^3} x_{SL} - \frac{Gm_\oplus m_S}{r_{\oplus S}^3} x_{\oplus S} - C_S \left[\left(\frac{5u_S^2}{r_{\oplus S}^2} - 1 \right) x_{\oplus S} - 2u_S \sin \theta \sin \phi \right]$
 14. $\dot{p}_{y_S} = \frac{Gm_S m_L}{r_{SL}^3} y_{SL} - \frac{Gm_\oplus m_S}{r_{\oplus S}^3} y_{\oplus S} - C_S \left[\left(\frac{5u_S^2}{r_{\oplus S}^2} - 1 \right) y_{\oplus S} + 2u_S \sin \theta \cos \phi \right]$
 15. $\dot{p}_{z_S} = \frac{Gm_S m_L}{r_{SL}^3} z_{SL} - \frac{Gm_\oplus m_S}{r_{\oplus S}^3} z_{\oplus S} - C_S \left[\left(\frac{5u_S^2}{r_{\oplus S}^2} - 1 \right) z_{\oplus S} - 2u_S \cos \theta \right]$
 16. $\dot{p}_{x_\oplus} = \frac{Gm_\oplus m_L}{r_{\oplus L}^3} x_{\oplus L} + C_L \left[\left(\frac{5u_L^2}{r_{\oplus L}^2} - 1 \right) x_{\oplus L} - 2u_L \sin \theta \sin \phi \right] + \frac{Gm_\oplus m_S}{r_{\oplus S}^3} x_{\oplus S} + C_S \left[\left(\frac{5u_S^2}{r_{\oplus S}^2} - 1 \right) x_{\oplus S} - 2u_S \sin \theta \sin \phi \right]$
 17. $\dot{p}_{y_\oplus} = \frac{Gm_\oplus m_L}{r_{\oplus L}^3} y_{\oplus L} + C_L \left[\left(\frac{5u_L^2}{r_{\oplus L}^2} - 1 \right) y_{\oplus L} + 2u_L \sin \theta \cos \phi \right] + \frac{Gm_\oplus m_S}{r_{\oplus S}^3} y_{\oplus S} + C_S \left[\left(\frac{5u_S^2}{r_{\oplus S}^2} - 1 \right) y_{\oplus S} + 2u_S \sin \theta \cos \phi \right]$
 18. $\dot{p}_{z_\oplus} = \frac{Gm_\oplus m_L}{r_{\oplus L}^3} z_{\oplus L} + C_L \left[\left(\frac{5u_L^2}{r_{\oplus L}^2} - 1 \right) z_{\oplus L} - 2u_L \cos \theta \right] + \frac{Gm_\oplus m_S}{r_{\oplus S}^3} z_{\oplus S} + C_S \left[\left(\frac{5u_S^2}{r_{\oplus S}^2} - 1 \right) z_{\oplus S} - 2u_S \cos \theta \right]$
 19. $\dot{p}_{x_L} = -\frac{Gm_S m_L}{r_{SL}^3} x_{SL} - \frac{Gm_\oplus m_L}{r_{\oplus L}^3} x_{\oplus L} - C_L \left[\left(\frac{5u_L^2}{r_{\oplus L}^2} - 1 \right) x_{\oplus L} - 2u_L \sin \theta \sin \phi \right]$
 20. $\dot{p}_{y_L} = -\frac{Gm_S m_L}{r_{SL}^3} y_{SL} - \frac{Gm_\oplus m_L}{r_{\oplus L}^3} y_{\oplus L} - C_L \left[\left(\frac{5u_L^2}{r_{\oplus L}^2} - 1 \right) y_{\oplus L} + 2u_L \sin \theta \cos \phi \right]$
 21. $\dot{p}_{z_L} = -\frac{Gm_S m_L}{r_{SL}^3} z_{SL} - \frac{Gm_\oplus m_L}{r_{\oplus L}^3} z_{\oplus L} - C_L \left[\left(\frac{5u_L^2}{r_{\oplus L}^2} - 1 \right) z_{\oplus L} - 2u_L \cos \theta \right]$
 22. $\dot{p}_\theta = \frac{(p_\phi - p_\psi \cos \theta)^2 \cos \theta}{I_\perp \sin^3 \theta} - \frac{(p_\phi - p_\psi \cos \theta) p_\psi}{I_\perp \sin \theta} + 2C_L u_L u_{\theta L} + 2C_S u_S u_{\theta S}$
 23. $\dot{p}_\phi = 2C_L u_L u_{\phi L} + 2C_S u_S u_{\phi S}$
 24. $\dot{p}_\psi = 0$
-

Al pasarlo a Python, primero pensé en resolverlo usando el método de Runge-Kutta 4. Pero, por lo que he visto, no es recomendable para sistemas donde el espacio fásico se conserva.

Sabemos que, por el Teorema de Louville, el volumen del espacio fásico se conserva en el tiempo. Por un motivo, RK4 hace que no se conserve el espacio fásico. El método RK4 presenta una deriva secular, que es un error que crece de forma monótona. Entonces, a lo largo del tiempo perderíamos precisión. Sin embargo, hay otros métodos para resolver las EDs, que son los integradores simplécticos. En este caso, la transformación es canónica.

Vamos a explicar bien el método de RK4 para ver donde nos falla y luego los métodos de integradores simplécticos para ver por qué estos sí que nos sirven.

RK4 vs Störmer-Verlet: Integradores Numéricos

Para comprobar por qué tenemos que usar un método de integradores simplécticos, vamos a usar la ecuación de un oscilador armónico simple. Esta demostración la he obtenido del libro "GEOMETRIC NUMERICAL INTEGRATION STRUCTURE-PRESERVING ALGORITHMS FOR ORDINARY DIFFERENTIAL EQUATIONS SECOND EDITION - Hairer".

Es un trabajo para entenderlo mientras intento explicar el procedimiento y todas las dudas que me van surgiendo mientras lo estudio.

¿Qué estamos resolviendo?

Vamos a simular la ecuación de un oscilador armónico simple:

$$\ddot{x} = -\omega^2 x$$

Para resolverla numéricamente, transformamos esta ecuación de segundo orden en un sistema de ecuaciones de primer orden. Si definimos la velocidad como $y = \dot{x}$, el sistema se escribe en forma vectorial como:

$$\frac{d}{dt}(x \ y) = (y \ -\omega^2 x)$$

donde x es la posición e y es la velocidad ($\dot{x}$). La energía (que es una cantidad conservada) es:

$$E = \frac{1}{2}(y^2 + \omega^2 x^2)$$

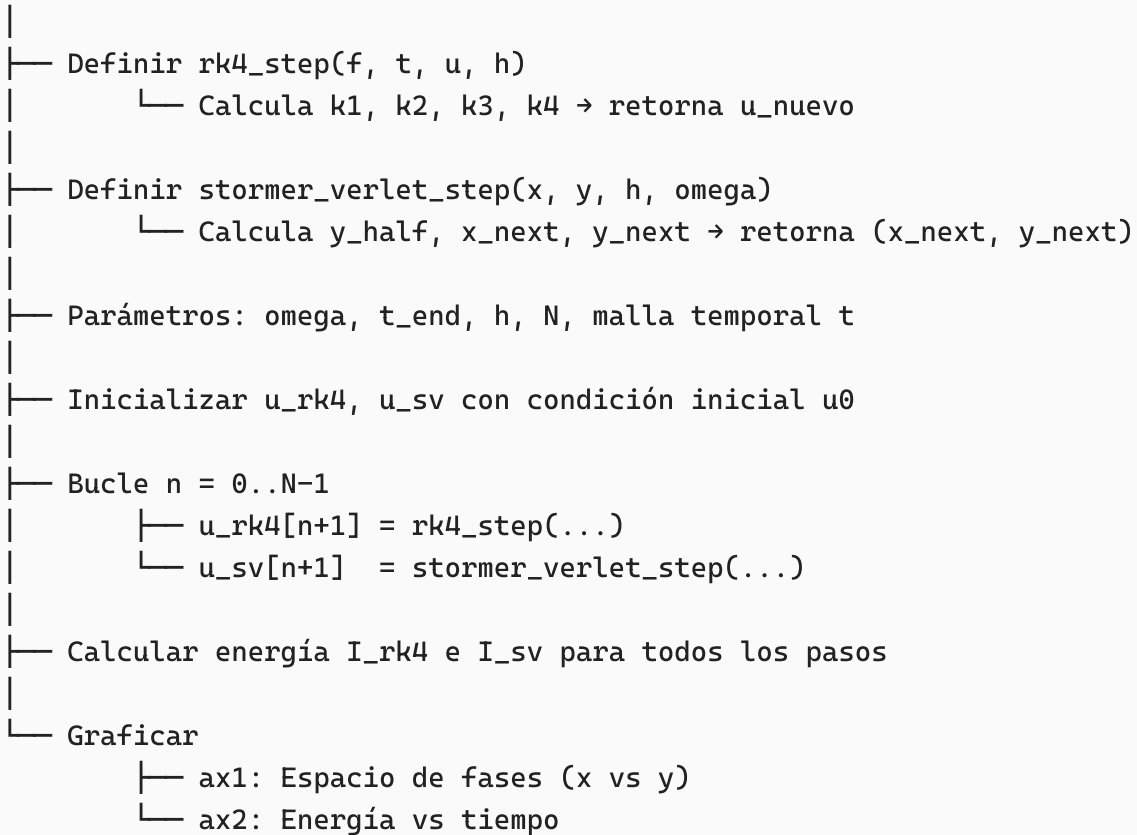
Hemos buscado una cantidad conservada porque así podemos ver si se conserva o no. Que es lo que luego usaremos para calcular las órbitas. Queremos que la E se conserve. Por aclararlo, esta fórmula de la E sale de $T + V$.

La pregunta central es: **¿cuánto tiempo conserva la energía cada integrador?**

Explicación del código

Diagrama del flujo

Importaciones



Importaciones

```
import numpy as np
import matplotlib.pyplot as plt
```

Librería	Función
numpy	Operaciones numéricas vectorizadas (arrays, operaciones matemáticas).
matplotlib.pyplot	Visualización: crear gráficas 2D.

Importamos numpy por que lo necesitamos para hacer las operaciones con vectores, lo que en python se llamarían arrays. Es la librería estándar para este tipo de cálculos. Luego, matplotlib lo usamos para graficar la energía respecto del tiempo. Lo mismo, librería estándar.

Sección 1: Definición de los Integradores

1.1 Runge-Kutta de Orden 4 (RK4)

```
def rk4_step(f, t, u, h):  
    """Avanza un paso 'h' usando el método de Runge-Kutta 4 estándar."""  
    k1 = f(t, u)  
    k2 = f(t + h/2, u + h*k1/2)  
    k3 = f(t + h/2, u + h*k2/2)  
    k4 = f(t + h, u + h*k3)  
    return u + (h/6)*(k1 + 2*k2 + 2*k3 + k4)
```

La fórmula de actualización:

$$u_{n+1} = u_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

Los pesos $\frac{1}{6}, \frac{2}{6}, \frac{2}{6}, \frac{1}{6}$ son exactamente los pesos de la regla de Simpson, que es una fórmula que se usa en el cálculo de integrales en cálculo numérico.

 **Orden de error: RK4 tiene error local $\mathcal{O}(h^5)$ por paso y error global $\mathcal{O}(h^4)$. Es muy preciso para pasos pequeños, pero no es simpléctico: no conserva la estructura geométrica del sistema hamiltoniano.**

1.2 Störmer-Verlet (Integrador Simpléctico)

```
def stormer_verlet_step(x_n, y_n, h, omega):  
    """Avanza un paso 'h' usando el esquema simpléctico de Störmer-Verlet."""  
    y_half = y_n - (h/2) * (omega**2) * x_n  
    x_next = x_n + h * y_half  
    y_next = y_half - (h/2) * (omega**2) * x_next  
    return x_next, y_next
```

¿Qué es un integrador simpléctico?

Un integrador **simpléctico** preserva el área en el espacio de fases (x, y) . Esto es consecuencia del **Teorema de Liouville** en mecánica hamiltoniana: los flujos hamiltonianos preservan el volumen (y el área en 2D).

Lo que esto quiere decir es que el integrador no aumenta ni disminuye artificialmente las trayectorias. La energía oscila alrededor del valor correcto en lugar de crecer o decrecer.

Las tres líneas del algoritmo

El esquema es un método *splitting*: se divide el hamiltoniano y se resuelve cada parte exactamente.

Paso 1: Medio paso de velocidad con la posición *actual*

$$y_{n+1/2} = y_n - \frac{h}{2} \omega^2 x_n$$

Paso 2: Paso completo de posición con la velocidad *a mitad*

$$x_{n+1} = x_n + h \cdot y_{n+1/2}$$

Paso 3: Medio paso de velocidad con la posición *nueva*

$$y_{n+1} = y_{n+1/2} - \frac{h}{2} \omega^2 x_{n+1}$$

🔥 El orden importa: el paso de posición usa la velocidad actualizada a mitad, y el segundo medio paso de velocidad usa la posición ya actualizada. Esta asimetría crea la simetría temporal que hace al método simpléctico.

Sección 2: Parámetros del Sistema

```
omega = 1.0
def sistema(t, u):
    x, y = u[0], u[1]
    return np.array([y, -omega**2 * x])

t0 = 0.0
t_end = 200.0 # Tiempo muy largo
h = 1.0       # PASO MUY GRANDE (pero estable para SV: |h*omega| = 1.0 <= 2.0)
N = int(np.ceil((t_end - t0)/h))
t = np.linspace(t0, t_end, N+1)
```

¿Por qué estos valores son extremos?

Parámetro	Valor	Razón
omega	1.0	Frecuencia natural del oscilador (período $2\pi \approx 6.28$)
t_end	200.0	Muy largo: ≈ 32 periodos completos. Las diferencias acumulan.
h	1.0	$\sim 1/6$ del período. El paso es muy grande para RK4; Störmer-Verlet es estable hasta $h\omega \leq 2$.

La función `sistema`

```
def sistema(t, u):
    x, y = u[0], u[1]
    return np.array([y, -omega**2 * x])
```

Creamos una función `sistema` con dos variables, t y u . t la usamos para marcar el paso del tiempo, y u como vector que estará compuesto por x e y .

Esta función codifica el lado derecho del sistema de EDOs:

$$f(t, u) = (y - \omega^2 x)$$

`u[0]` es x (posición), `u[1]` es y (velocidad). Retorna un array de NumPy para poder hacer aritmética vectorizada con él.

Construcción de la malla temporal

```
N = int(np.ceil((t_end - t0)/h)) # Número de pasos
t = np.linspace(t0, t_end, N+1)  # N+1 puntos: t0, t1, ..., tN
```

`np.ceil` redondea ese número al entero superior más cercano, de esa forma, podemos encontrar mejor los pasos necesarios. `np.linspace(a, b, n)` genera n puntos igualmente espaciados de a a b , **incluyendo ambos extremos**.

Sección 3: Inicialización y Condiciones Iniciales

```
u_rk4 = np.zeros((N+1, 2))
u_sv = np.zeros((N+1, 2))

u0 = np.array([1.0, 0.0])
u_rk4[0] = u0
u_sv[0] = u0
```

`np.zeros((N+1, 2))` crea una matriz de ceros con filas $N + 1$ instantes de tiempo (uno por cada paso, más el inicial) y columnas con 2 variables del estado (x, y) .

☰ Ejemplo de estructura

```
u_rk4[0] = [x0, y0] ← condición inicial
u_rk4[1] = [x1, y1] ← después del primer paso
u_rk4[N] = [xn, yn] ← estado final
```

Lo que estaríamos haciendo ahora sería esto. Creamos la matriz (imaginemos que solo vamos a realizar 3 pasos, entonces, necesito 4 filas):

$$U = (0.0 \quad 0.0 \ 0.0 \quad 0.0 \ 0.0 \quad 0.0 \ 0.0) \leftarrow t_0 \leftarrow t_1 \leftarrow t_2 \leftarrow t_3$$

La primera columna es la posición, la segunda las velocidades. Digamos que la posición inicial es 1.0:

$$U = (1.0 \quad 0.0 \ 0.0 \quad 0.0 \ 0.0 \quad 0.0 \ 0.0)$$

Se ejecuta el algoritmo del Störmer-Verlet:

- **Paso 1 (Medio paso de velocidad):** $y_{1/2} = y_0 - \frac{h}{2}\omega^2 x_0 = 0.0 - \frac{1.0}{2}(1.0)^2(1.0) = -0.5$
- **Paso 2 (Paso completo de posición):** $x_1 = x_0 + h y_{1/2} = 1.0 + 1.0(-0.5) = 0.5$
- **Paso 3 (Medio paso de velocidad restante):**

$$y_1 = y_{1/2} - \frac{h}{2}\omega^2 x_1 = -0.5 - \frac{1.0}{2}(1.0)^2(0.5) = -0.75$$

$$U = (1.0 \quad 0.0 \ 0.5 \quad -0.75 \ 0.0 \quad 0.0 \ 0.0)$$

Y seguiría así hasta terminar.

Con $x_0 = 1$ e $y_0 = 0$ (partícula en reposo en $x = 1$):

$$I_0 = \frac{1}{2}(0^2 + 1^2 \cdot 1^2) = 0.5$$

Ésta es la energía que **debería conservarse** durante toda la simulación.

Sección 4: Bucle de Integración

```
for n in range(N):
    # Paso de RK4
    u_rk4[n+1] = rk4_step(sistema, t[n], u_rk4[n], h)
```

```
# Paso de Störmer-Verlet
x_next, y_next = stormer_verlet_step(u_sv[n, 0], u_sv[n, 1], h, omega)
u_sv[n+1, 0] = x_next
u_sv[n+1, 1] = y_next
```

Itera desde el paso $n = 0$ hasta $n = N - 1$ (es decir, N pasos en total), calculando el estado en t_{n+1} a partir del estado en t_n .

Para RK4 basta pasar el array de estado completo. Para Störmer-Verlet extraemos x e y por separado porque el algoritmo opera sobre posición y velocidad de forma explícita.

🔗 `u_sv[n, 0]` accede a la fila n , columna 0 (la posición x en el paso n). `u_sv[n]` devolvería el array completo $[x_n, y_n]$.

Sección 5: Análisis de Energía

```
I_rk4 = 0.5 * (u_rk4[:, 1]**2 + omega**2 * u_rk4[:, 0]**2)
I_sv = 0.5 * (u_sv[:, 1]**2 + omega**2 * u_sv[:, 0]**2)
```

`u_rk4[:, 0]` extrae **todas las filas, columna 0** → array con todos los valores de x . `u_rk4[:, 1]` extrae **todas las filas, columna 1** → array con todos los valores de y . La operación `**2` eleva al cuadrado **elemento a elemento** (NumPy broadcasting).

Resultado: `I_rk4` e `I_sv` son arrays de $N + 1$ valores de energía, uno por cada instante de tiempo. La energía exacta del oscilador armónico es:

$$I(t) = \frac{1}{2}(\dot{x}^2 + \omega^2 x^2) = \frac{1}{2}(y^2 + \omega^2 x^2) = \text{constante}$$

Sección 6: Visualización

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))
```

`plt.subplots(1, 2)` crea una figura con **1 fila, 2 columnas** de subgráficas. Retorna `fig` (el objeto figura completo) y `(ax1, ax2)` (los dos ejes, unpacking de tupla).

Gráfica 1 — Espacio de Fases

```
ax1.plot(u_rk4[:, 0], u_rk4[:, 1], 'r-',
        label='RK4 (Pérdida de Área Catastrófica)', alpha=0.9, linewidth=1.0)
ax1.plot(u_sv[:, 0], u_sv[:, 1], 'b-',
        label='Störmer-Verlet (Conservación del Área)', alpha=0.6,
        linewidth=1.5)
ax1.axis('equal')
```

El espacio de fases grafica x vs y . Para el oscilador armónico exacto, la trayectoria es una **elipse perfecta** (o círculo si $\omega = 1$).

Comportamiento esperado	RK4	Störmer-Verlet
Trayectoria en espacio de fases	Espiral hacia el origen (pierde energía)	Elipse cerrada (conserva área)

`ax1.axis('equal')` fuerza la misma escala en ambos ejes para que una elipse se vea como elipse y no deformada.

Gráfica 2 — Evolución de Energía

```
ax2.plot(t, I_rk4, 'r-', label='RK4 (Disipación Numérica Completa)',
        linewidth=2.0)
ax2.plot(t, I_sv, 'b-', label='Störmer-Verlet', linewidth=2.0)
ax2.set_ylim(-0.05, 0.55)
```

`ax2.set_ylim(-0.05, 0.55)` fuerza el rango del eje y para que veamos el **colapso a 0** de RK4 de forma dramática. Sin esto, la escala se ajustaría automáticamente y podría ocultar el comportamiento.

¿Qué resultados esperamos ver?

RK4: Disipación numérica

RK4 es un método **disipativo**: tiene una región de estabilidad finita. Con $h = 1.0$ y $\omega = 1.0$, el valor $h\omega = 1.0$ está cerca del límite de estabilidad. RK4 introduce amortiguamiento artificial que reduce la energía monótonamente, colapsa la trayectoria en espacio de fases hacia el origen (espiral) y hace que $I_{rk4}(t) \rightarrow 0$.

El motivo de por qué ocurre esto es el siguiente. El sistema de ecuaciones de primer orden del oscilador armónico se puede escribir en forma matricial como $\dot{\mathbf{u}} = A\mathbf{u}$, donde:

$$A = \begin{pmatrix} 0 & 1 & -\omega^2 & 0 \end{pmatrix}$$

La solución viene dada por $\mathbf{u}(t+h) = e^{hA}\mathbf{u}(t)$. El método RK4 es, en el fondo, una aproximación de la serie de Taylor de la función exponencial truncada hasta el cuarto orden. Aplicado a nuestro sistema lineal $\dot{\mathbf{u}} = A\mathbf{u}$, la matriz de avance numérico M_{RK4} es:

$$M_{RK4} = I + hA + \frac{h^2}{2!}A^2 + \frac{h^3}{3!}A^3 + \frac{h^4}{4!}A^4$$

Para simplificar el cálculo y ver el efecto neto, asumamos $\omega = 1$. Calculando las potencias de la matriz A :

- $A^2 = \begin{pmatrix} -1 & 0 & 0 & -1 \end{pmatrix} = -I$
- $A^3 = -A = \begin{pmatrix} 0 & -1 & 1 & 0 \end{pmatrix}$
- $A^4 = I = \begin{pmatrix} 1 & 0 & 0 & 1 \end{pmatrix}$

Sustituyendo en la ecuación de M_{RK4} y agrupando los términos:

$$M_{RK4} = \left(1 - \frac{h^2}{2} + \frac{h^4}{24}\right)I + \left(h - \frac{h^3}{6}\right)A$$

Lo que nos deja la siguiente matriz de transición para cada paso de tiempo en RK4:

$$M_{RK4} = \begin{pmatrix} 1 - \frac{h^2}{2} + \frac{h^4}{24} & h - \frac{h^3}{6} & -h + \frac{h^3}{6} & 1 - \frac{h^2}{2} + \frac{h^4}{24} \end{pmatrix}$$

Para saber si RK4 conserva el área del espacio de fases, calculamos el determinante de M_{RK4} :

$$\det(M_{RK4}) = \left(1 - \frac{h^2}{2} + \frac{h^4}{24}\right)^2 + \left(h - \frac{h^3}{6}\right)^2$$

Si desarrollas algebraicamente este polinomio, los términos hasta h^4 se cancelan perfectamente, pero el polinomio completo resultante es:

$$\det(M_{RK4}) = 1 - \frac{h^6}{72} + \frac{h^8}{576}$$

Störmer-Verlet: Conservación simpléctica

Störmer-Verlet es estable para $|h\omega| \leq 2$. Con $h\omega = 1.0$, la energía **oscila** alrededor de 0.5 pero **no deriva** a largo plazo. La trayectoria en espacio de fases es una elipse cerrada o casi cerrada. No hay amortiguamiento artificial: el método "respeto" la geometría hamiltoniana.

1. Descomposición matricial del algoritmo

El método de Störmer-Verlet se compone de tres pasos. Vamos a escribir la matriz correspondiente a cada uno de ellos:

Paso 1: Medio paso de velocidad

La ecuación es $y_{n+1/2} = y_n - \frac{h}{2}\omega^2 x_n$. La posición no cambia ($x_n = x_n$).

Matricialmente, esto es:

$$\begin{pmatrix} x_n & y_{n+1/2} \end{pmatrix} = \begin{pmatrix} 1 & 0 & -\frac{h}{2}\omega^2 & 1 \end{pmatrix} \begin{pmatrix} x_n & y_n \end{pmatrix}$$

Llamemos a esta primera matriz M_1 .

Paso 2: Paso completo de posición

La ecuación es $x_{n+1} = x_n + hy_{n+1/2}$. La velocidad a medio paso se mantiene intacta para esta operación.

Matricialmente:

$$\begin{pmatrix} x_{n+1} & y_{n+1/2} \end{pmatrix} = \begin{pmatrix} 1 & h & 0 & 1 \end{pmatrix} \begin{pmatrix} x_n & y_{n+1/2} \end{pmatrix}$$

Llamemos a esta matriz M_2 .

Paso 3: Medio paso de velocidad restante

La ecuación es $y_{n+1} = y_{n+1/2} - \frac{h}{2}\omega^2 x_{n+1}$. La posición ya actualizada se mantiene.

Matricialmente:

$$\begin{pmatrix} x_{n+1} & y_{n+1} \end{pmatrix} = \begin{pmatrix} 1 & 0 & -\frac{h}{2}\omega^2 & 1 \end{pmatrix} \begin{pmatrix} x_{n+1} & y_{n+1/2} \end{pmatrix}$$

Llamemos a esta matriz M_3 (nota que M_3 es exactamente idéntica a M_1).

2. La matriz de transición global (M_{SV})

El avance completo de un paso temporal $\mathbf{u}_{n+1} = M_{SV}\mathbf{u}_n$ se obtiene multiplicando estas tres matrices en el orden en que se aplican (de derecha a izquierda):

$$M_{SV} = M_3 M_2 M_1$$

Primero calculamos $M_2 M_1$:

$$M_2 M_1 = \begin{pmatrix} 1 & h & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & -\frac{h}{2}\omega^2 & 1 \end{pmatrix} = \begin{pmatrix} 1 - \frac{h^2}{2}\omega^2 & h & -\frac{h}{2}\omega^2 & 1 \end{pmatrix}$$

Ahora multiplicamos M_3 por el resultado anterior para obtener la matriz final M_{SV} :

$$M_{SV} = \begin{pmatrix} 1 & 0 & -\frac{h}{2}\omega^2 & 1 \end{pmatrix} \begin{pmatrix} 1 - \frac{h^2}{2}\omega^2 & h & -\frac{h}{2}\omega^2 & 1 \end{pmatrix}$$

Operando fila por columna, obtenemos la matriz de transición exacta de Störmer-Verlet:

$$M_{SV} = \begin{pmatrix} 1 - \frac{h^2}{2}\omega^2 & h & -h\omega^2 + \frac{h^3}{4}\omega^4 & 1 - \frac{h^2}{2}\omega^2 \end{pmatrix}$$

Para comprobar la simplecticidad, calculamos el determinante:

$$\det(M_{SV}) = \left(1 - \frac{h^2}{2}\omega^2\right)^2 - (h) \left(-h\omega^2 + \frac{h^3}{4}\omega^4\right)$$

Desarrollando:

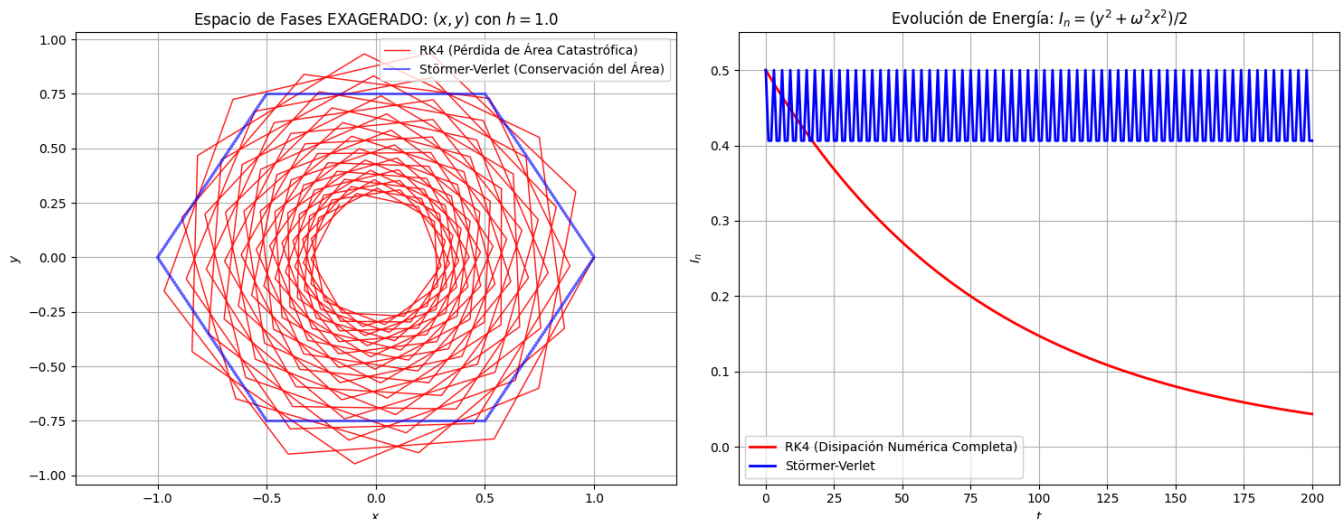
- El cuadrado: $1 - h^2\omega^2 + \frac{h^4}{4}\omega^4$
- El segundo término: $h^2\omega^2 - \frac{h^4}{4}\omega^4$

Sumando, todos los términos que dependen de h y ω se cancelan algebraicamente de forma exacta:

$$\det(M_{SV}) = 1$$

A diferencia de RK4, donde el determinante era una serie truncada menor a 1 que comprimía el área paso tras paso, en Störmer-Verlet **la transformación matricial tiene un determinante idéntico a 1 por construcción**. Esto demuestra algebraicamente que, sin importar el tamaño del paso h , el método no introducirá ninguna contracción geométrica (amortiguamiento) en la matriz de evolución temporal.

Aquí vemos las soluciones que ha dado el programa. Como podemos apreciar, la energía en el RK4 desciende mientras que en Störmer-Verlet oscila alrededor de un mismo valor, manteniéndose constante.



Benchmarks

Me he dado cuenta de que voy a querer construir varios modelos, porque, por ejemplo, tendré que construir un modelo con el resto de planetas del sistema solar buscando que sea más preciso.

Para ello, he creado un programa que nos sirva para clasificar después los programas:

```
import numpy as np
import json
import os
import argparse
from datetime import datetime, timedelta
```

Primero, introducimos las bibliotecas necesarias: Numpy ya hemos comentado antes que es para facilitar los cálculos. Os es para verificar que los imports se realizan correctamente. No es estrictamente necesario, pero como he tenido algún problema otra vez al realizar imports, lo he metido. Datetime y timedelta nos sirve para marcar el tiempo del proceso y el paso del tiempo.

Constantes y fecha del eclipse

```
AU = 1.495978707e11; DAY = 86400.0; KM = 1e3
R_SOL = 6.957e8 / AU; R_LUNA = 1.7374e6 / AU; R_TIERRA = 6.371e6 / AU
ECLIPSE.UTC = datetime(2026, 8, 12, 17, 47, 6)
REFERENCE_FILE = "Datos/jpl_reference_data.json"
```

Datos de referencia que usaremos para comparar como de cerca se han quedado nuestras predicciones.

benchmark.py

```
"""
=====
=
BENCHMARK – Proyecto Eclipse Solar Total 12 agosto 2026
=====
=

Estructura del archivo:
1. fetch_reference_data() → descarga datos JPL Horizons (ejecutar una vez)
2. load_reference_data() → carga los datos guardados
3. eclipse_error() → calcula las 3 métricas de error para cualquier
modelo
4. print_benchmark() → tabla comparativa de todos los modelos
```

Uso:

```
python benchmark.py --fetch      # descargar datos de referencia de JPL
python benchmark.py --check      # verificar datos descargados
python benchmark.py --demo       # mostrar tabla de ejemplo con un modelo
```

dummy

"""

```
import numpy as np
import json
import os
import argparse
from datetime import datetime, timedelta
```

——— CONSTANTES

```
AU          = 1.495978707e11    # metros
DAY          = 86400.0          # segundos
KM           = 1e3              # metros
ARCMIN       = 1/60.0           # grados
```

```
R_SOL        = 6.957e8 / AU
R_LUNA       = 1.7374e6 / AU
R_TIERRA     = 6.371e6 / AU
```

```
ECLIPSE.UTC = datetime(2026, 8, 12, 17, 47, 6)
```

```
FECHAS_CONTROL = [
    datetime(2026, 1, 1, 0, 0, 0),
    datetime(2026, 3, 1, 0, 0, 0),
    datetime(2026, 6, 1, 0, 0, 0),
    datetime(2026, 8, 1, 0, 0, 0),
    datetime(2026, 8, 12, 17, 47, 6),
]
```

```
REFERENCE_FILE = "Datos/jpl_reference_data.json"
```

——— 1. DESCARGA DE DATOS DE REFERENCIA

```
def fetch_reference_data():
    try:
        from astroquery.jplhorizons import Horizons
        print("Usando astroquery.jplhorizons")
        return _fetch_con_astroquery(Horizons)
    except ImportError:
        print("astroquery no disponible, usando API REST directa.")
```

```

try:
    import requests
except ImportError:
    print("ERROR: instala requests:  pip install requests")
    return False
return _fetch_con_requests(requests)

def _fetch_con_astroquery(Horizons):
    cuerpos = [("10", "Sol"), ("399", "Tierra"), ("301", "Luna"),
("199", "Mercurio"),
                ("299", "Venus"), ("499", "Marte"), ("599", "Jupiter"),
("699", "Saturno"),
                ("799", "Urano"), ("899", "Neptuno")]

    datos = {}
    for fecha in FECHAS_CONTROL:
        from astropy.time import Time
        t = Time(fecha.isoformat(), format='isot', scale='utc')
        print(f"\nFecha: {fecha.strftime('%Y-%b-%d %H:%M')}")
        datos[fecha.isoformat()] = {}
        for jpl_id, nombre in cuerpos:
            try:
                obj = Horizons(id=jpl_id, location="500@0", epochs=t.jd,
id_type="majorbody")
                vecs = obj.vectors(refplane="ecliptic")
                x,y,z = float(vecs["x"][0]),float(vecs["y"]
[0]),float(vecs["z"][0])
                vx,vy,vz= float(vecs["vx"][0]),float(vecs["vy"]
[0]),float(vecs["vz"][0])
                r = np.sqrt(x**2+y**2+z**2)
                datos[fecha.isoformat()][nombre] = {"pos": [x,y,z], "vel":
[vx,vy,vz], "r": float(r)}
                print(f" {nombre}: r={r:.6f} AU OK")
            except Exception as e:
                print(f" {nombre}: ERROR - {e}")
        with open(REFERENCE_FILE, "w") as f: json.dump(datos, f, indent=2)
        print(f"\n/ Datos guardados en {REFERENCE_FILE}")
    return True

def _fetch_con_requests(requests):
    BASE_URL = "https://ssd.jpl.nasa.gov/api/horizons.api"
    cuerpos = [("10", "Sol"), ("399", "Tierra"), ("301", "Luna"),
("199", "Mercurio"),
                ("299", "Venus"), ("499", "Marte"), ("599", "Jupiter"),
("699", "Saturno"),

```

```

        ("799", "Urano"), ("899", "Neptuno")]
    datos = {}
    for fecha in FECHAS_CONTROL:
        fecha_str = fecha.strftime("%Y-%b-%d %H:%M")
        fecha_fin_str = (fecha+timedelta(minutes=2)).strftime("%Y-%b-%d
%H:%M")
        print(f"\nFecha: {fecha_str}")
        datos[fecha.isoformat()] = {}
        for jpl_id, nombre in cuerpos:
            params =
{"format": "text", "COMMAND": jpl_id, "OBJ_DATA": "NO", "MAKE_EPHEM": "YES",

"EPHEM_TYPE": "VECTORS", "CENTER": "500@0", "START_TIME": fecha_str,
"STOP_TIME": fecha_fin_str, "STEP_SIZE": "1
m", "VEC_TABLE": "2",

"REF_PLANE": "ECLIPTIC", "REF_SYSTEM": "J2000", "OUT_UNITS": "AU-D",
"VEC_LABELS": "YES", "CSV_FORMAT": "NO"}

        try:
            r = requests.get(BASE_URL, params=params, timeout=30)
            r.raise_for_status()
            rv = _parse_horizons_texto(r.text)
            if rv:
                datos[fecha.isoformat()][nombre] = rv
                print(f" {nombre}: r={rv['r']:.6f} AU OK")
            else:
                print(f" {nombre}: ERROR al parsear")
        except Exception as e:
            print(f" {nombre}: EXCEPCION - {e}")
    with open(REFERENCE_FILE, "w") as f: json.dump(datos, f, indent=2)
    print(f"\n/ Datos guardados en {REFERENCE_FILE}")
    return True

def _parse_horizons_texto(text):
    import re
    pos = None; vel = None
    for line in text.split("\n"):
        line_s = line.strip()
        if not line_s: continue
        if pos is None and re.search(r'\bX\s*=', line_s) and
re.search(r'\bY\s*=', line_s):
            nums = re.findall(r'[XYZ]\s*=\s*([+-]?[d+\.\d+[Ee][+-]?
\d+)', line_s)
            if len(nums)>=3: pos=
[float(nums[0]), float(nums[1]), float(nums[2])]

```

```

        if vel is None and re.search(r'\bVX\s*=', line_s) and
re.search(r'\bVY\s*=', line_s):
            nums = re.findall(r'V[XYZ]\s*=\s*([+-]?\d+\.\d+[Ee][+-]?
\d+)', line_s)
            if len(nums)>=3: vel=
[float(nums[0]),float(nums[1]),float(nums[2])]
            if pos is not None and vel is not None: break
            if pos is not None and vel is not None:
                return {"pos":pos, "vel":vel, "r":float(np.linalg.norm(pos))}
            return None

```

— 2. CARGA DE DATOS DE REFERENCIA

```

def load_reference_data():
    if not os.path.exists(REFERENCE_FILE):
        print(f"ERROR: no existe {REFERENCE_FILE}. Ejecuta: python
benchmark.py --fetch")
        return None
    with open(REFERENCE_FILE) as f: datos = json.load(f)
    print(f"✓ Datos de referencia cargados: {len(datos)} fechas ×
{len(list(datos.values())[0])} cuerpos")
    return datos

```

— 3. CÁLCULO DE ERROR

```

def eclipse_error(pos_sol, pos_tierra, pos_luna, t_eclipse_pred=None,
referencia=None):
    pos_sol=np.array(pos_sol); pos_tierra=np.array(pos_tierra);
pos_luna=np.array(pos_luna)
    v_tl=pos_luna-pos_tierra; v_ts=pos_sol-pos_tierra
    d_luna=np.linalg.norm(v_tl); d_sol=np.linalg.norm(v_ts)
    cos_sep=np.clip(np.dot(v_ts,v_tl)/(d_sol*d_luna),-1,1)
    sep_grados=np.degrees(np.arccos(cos_sep)); sep_arcmin=sep_grados*60.0
    r_ang_sol=np.degrees(np.arcsin(np.clip(R_SOL/d_sol,0,1)))*60
    r_ang_luna=np.degrees(np.arcsin(np.clip(R_LUNA/d_luna,0,1)))*60
    fase_magnitud=r_ang_luna/r_ang_sol
    es_total=(fase_magnitud>1.0) and (sep_arcmin<r_ang_sol)
    if referencia is not None:
        ref_luna=np.array(referencia["Luna"]["pos"]);
ref_tierra=np.array(referencia["Tierra"]["pos"])
        error_luna_km=np.linalg.norm((pos_luna-pos_tierra)-(ref_luna-
ref_tierra))*AU/KM

```

```

        ref_sol=np.array(referencia["Sol"]["pos"]); v_rl=ref_luna-ref_tierra;
v_rs=ref_sol-ref_tierra

cos_ref=np.clip(np.dot(v_rs,v_rl)/(np.linalg.norm(v_rs)*np.linalg.norm(v_rl)),
-1,1)
        error_ang_arcmin=abs(sep_arcmin-np.degrees(np.arccos(cos_ref))*60.0)
    else:
        error_luna_km=None; error_ang_arcmin=None
        error_tiempo_min=abs((t_eclipse_pred-ECLIPSE.UTC).total_seconds()/60.0) if
t_eclipse_pred else None
        return
{"sep_sol_luna_arcmin":round(sep_arcmin,4),"r_ang_sol_arcmin":round(r_ang_sol,
4),

"r_ang_luna_arcmin":round(r_ang_luna,4),"fase_magnitud":round(fase_magnitud,5)
,
        "es_total":es_total,"d_luna_km":round(d_luna*A.U./KM),
        "error_pos_luna_km":round(error_luna_km) if error_luna_km is not
None else None,
        "error_ang_arcmin":round(error_ang_arcmin,3) if error_ang_arcmin
is not None else None,
        "error_tiempo_min":round(error_tiempo_min,1) if error_tiempo_min
is not None else None}

```

— 4. TABLA COMPARATIVA

```

def print_benchmark(resultados):
    sep="—" * 95
    print("\n\n┌" + "=" * 93 + "┐")
    print("┆ BENCHMARK – Eclipse Solar Total · 12 agosto 2026 · Error vs JPL
Horizons DE441" + " " * 11 + "┆")
    print("└" + "=" * 93 + "┘")
    print(f"\n{'Modelo':<30} {'d_Luna':>10} {'Err_pos':>12} {'Err_ang':>12}
{'Err_t':>8} {'Total?':>7} {'CPU':>6}")
    print(f"{'':<30} {'km':>10} {'km':>12} {'arcmin':>12} {'min':>8}
{'':>7} {'s':>6}")
    print(sep)
    for r in resultados:
        m=r["metricas"]
        ep=f"{'error_pos_luna_km':.0f}" if m["error_pos_luna_km"] is not
None else "—"
        ea=f"{'error_ang_arcmin':.3f}" if m["error_ang_arcmin"] is not
None else "—"
        et=f"{'error_tiempo_min':.1f}" if m["error_tiempo_min"] is not

```

```

None else "-
    tt="/ TOTAL" if m["es_total"] else "X anular"
    print(f"{r['nombre']:<30} {m['d_luna_km']:>10,} {ep:>12} {ea:>12}
{et:>8} {tt:>7} {r.get('tiempo_cpu_s',0):>6.1f}")
    print(sep)

if __name__=="__main__":
    parser=argparse.ArgumentParser()
    parser.add_argument("--fetch",action="store_true"); parser.add_argument("--
demo",action="store_true")
    parser.add_argument("--all",action="store_true"); args=parser.parse_args()
    if args.fetch: fetch_reference_data()
    elif args.demo: demo_con_v10()
    elif args.all: comparar_todos()
    else: parser.print_help()

```

Y, antes de empezar con los modelos, he creado un programa con las constantes que vamos a necesitar, para que los códigos estén más limpios:

```

"""
CONSTANTES FÍSICAS Y ASTRONÓMICAS
Base para todos los modelos del Eclipse 2026.
"""

import numpy as np

# ——— CONSTANTES FUNDAMENTALES


---


AU   = 1.495978707e11    # m
DAY  = 86400.0           # s
G    = 6.6743e-11 * DAY**2 / AU**3    # AU³ / (kg·día²)

# ——— CUERPOS Y MASAS


---


NOMBRES = ['Sol', 'Tierra', 'Luna', 'Mercurio', 'Venus', 'Marte',
           'Jupiter', 'Saturno', 'Urano', 'Neptuno']

MASAS = np.array([
    1.98892e30, # Sol
    5.97219e24, # Tierra
    7.34581e22, # Luna
    3.30200e23, # Mercurio
    4.86850e24, # Venus
    6.41712e23, # Marte

```



```

1.89813e27, # Jupiter
5.68319e26, # Saturno
8.68103e25, # Urano
1.02410e26, # Neptuno
])
N = len(MASAS)
GM = G * MASAS # AU³/día²

# Índices base
I_SOL = 0
I_TIE = 1
I_LUN = 2

# Masas específicas (legacy variable names para compatibilidad)
mS = MASAS[I_SOL]
mE = MASAS[I_TIE]
mL = MASAS[I_LUN]
GmS = GM[I_SOL]
GmE = GM[I_TIE]
GmL = GM[I_LUN]

# ——— MOMENTOS DE INERCIA (Tierra)


---


_m2AU2 = 1.0 / AU**2
I_perp = 8.0096e37 * _m2AU2
I_z     = 8.0358e37 * _m2AU2
dI      = I_z - I_perp

# ——— CONDICIONES INICIALES DE ROTACIÓN TERRESTRE


---


eps      = np.radians(23.4393) # oblicuidad J2000
theta_0  = eps                 #  $\theta$  inicial
phi_0    = 0.0                 #  $\varphi$  inicial (longitud del nodo ascendente)
psi_0    = 0.0                 #  $\psi$  inicial (ángulo de rotación propia)

omega_rot = 2 * np.pi / 0.9972697 # rad/día (período sidéreo)
phi_dot_0 = 0.0                    # precesión despreciable
psi_dot_0 = omega_rot - phi_dot_0 * np.cos(theta_0)

# Momentos conjugados iniciales de la rotación
ptheta_0 = I_perp * 0.0           #  $\dot{\theta} = 0$  (eje polar estable)
ppsi_0    = I_z * psi_dot_0       #  $p\psi = I_z \cdot (\dot{\psi} + \varphi \cos\theta)$ 
pphi_0    = I_perp * phi_dot_0 * np.sin(theta_0)**2 + ppsi_0 * np.cos(theta_0)

# ——— COEFICIENTES INTEGRADOR YOSHIDA O(4)


---



```

```

_w1 = 1.0 / (2.0 - 2.0**((1.0/3.0))
_w0 = -2.0**((1.0/3.0)) * _w1
C_YOSHIDA = np.array([_w1/2.0, (_w0+_w1)/2.0, (_w0+_w1)/2.0, _w1/2.0])
D_YOSHIDA = np.array([_w1, _w0, _w1])

```

Implementación del modelo

Una vez ya hemos visto por qué tenemos que usar modelos de integradores simplécticos para nuestro modelado, tenemos los benchmarks para comparar y las constantes, podemos proceder a aplicar nuestras ecuaciones del hamiltoniano a un código.

El método de integradores elegido es el **Método de Yoshida de orden 4**, que es una extensión del Störmer-Verlet que eleva el orden de exactitud a $\mathcal{O}(h^4)$ manteniendo la simpecticidad. El esquema tiene tres sub-pasos con coeficientes calculados para cancelar los términos de error hasta orden 4:

$$w_1 = \frac{1}{2 - 2^{1/3}}, \quad w_0 = -2^{1/3} \cdot w_1$$

Coeficientes de drift: $c = \left[\frac{w_1}{2}, \frac{w_0+w_1}{2}, \frac{w_0+w_1}{2}, \frac{w_1}{2} \right]$

Coeficientes de kick: $d = [w_1, w_0, w_1]$

Explicación del código

```

import numpy as np
import time
import sys
import os
from datetime import datetime, timedelta

```

Primero, introducimos las bibliotecas necesarias, que ya hemos explicado anteriormente.

```

IxS=0; IyS=1; IzS=2; IpxS=3; IpyS=4; IpzS=5
IxE=6; IyE=7; IzE=8; IpxE=9; IpyE=10; IpzE=11
IxE=12; IyL=13; IzL=14; IpxL=15; IpyL=16; IpzL=17
Ith=18; Iph=19; Ips=20; Ipth=21; Ipph=22; Ipps=23
N_STATE = 24

```

Aquí asignamos índices a diferentes variables que usaremos en este modelo.

I — — representan la posición en cada eje del cuerpo.

Ip — — representan el momento en cada eje del cuerpo.

La última línea hace referencia a los ángulos de Euler de la Tierra (18, 19, 20) y el resto a los momento angulares de la Tierra.

S es el Sol, E la Tierra y L la luna.

```
def derivadas(state):  
    xS, yS, zS = state[IxS], state[IyS], state[IzS]  
    pxS, pyS, pzS = state[IpXS], state[IpYS], state[IpzS]  
    xE, yE, zE = state[IxE], state[IyE], state[IzE]  
    pxE, pyE, pZE = state[IpxE], state[IpYE], state[IpzE]  
    xL, yL, zL = state[IxL], state[IyL], state[IzL]  
    pXL, pyL, pZL = state[IpXL], state[IpYL], state[IpzL]  
    th, ph, ps = state[Ith], state[Iph], state[Ips]  
    pth, pph, pps = state[Ipth], state[IpPh], state[IpPs]  
  
    x_EL = xL-xE; y_EL = yL-yE; z_EL = zL-zE  
    x_ES = xS-xE; y_ES = yS-yE; z_ES = zS-zE  
    x_SL = xL-xS; y_SL = yL-yS; z_SL = zL-zS
```

Aquí les asignamos unos nombres más legibles a las variables antes nombradas, para así poder trabajar mejor. Además, luego creamos las variables que nos dicen las distancias entre los cuerpos.

```
    r_EL2=x_EL**2+y_EL**2+z_EL**2; r_ES2=x_ES**2+y_ES**2+z_ES**2;  
r_SL2=x_SL**2+y_SL**2+z_SL**2  
    r_EL=np.sqrt(r_EL2); r_ES=np.sqrt(r_ES2); r_SL=np.sqrt(r_SL2)  
    r_EL3=r_EL2*r_EL; r_ES3=r_ES2*r_ES; r_SL3=r_SL2*r_SL  
    r_EL5=r_EL3*r_EL2; r_ES5=r_ES3*r_ES2
```

Precalculámos los radios para optimizar el código. Tenemos el radio al cuadrado, al cubo y a la quinta. Algunos de estos términos son por el cuadripotencial de la Tierra y no considerarla perfectamente esférica.

```
    sin_th=np.sin(th); cos_th=np.cos(th); sin_ph=np.sin(ph); cos_ph=np.cos(ph)  
    up_x=sin_th*sin_ph; up_y=-sin_th*cos_ph; up_z=cos_th  
    uL=x_EL*up_x+y_EL*up_y+z_EL*up_z; uS=x_ES*up_x+y_ES*up_y+z_ES*up_z  
    uθL=x_EL*cos_th*sin_ph-y_EL*cos_th*cos_ph-z_EL*sin_th  
    uφL=x_EL*sin_th*cos_ph+y_EL*sin_th*sin_ph  
    uθS=x_ES*cos_th*sin_ph-y_ES*cos_th*cos_ph-z_ES*sin_th  
    uφS=x_ES*sin_th*cos_ph+y_ES*sin_th*sin_ph
```

Calculamos los vectores unitarios de la Tierra (que es pasarlo a esféricas), la proyección del vector de la Luna para saber su altura y los vectores de la base local para saber como varía la energía con los ángulos.

```

IxS=0; IyS=1; IzS=2; IpxS=3; IpyS=4; IpzS=5
IxE=6; IyE=7; IzE=8; IpxE=9; IpyE=10; IpzE=11
IxL=12; IyL=13; IzL=14; IpxL=15; IpyL=16; IpzL=17
Ith=18; Iph=19; Ips=20; Ipth=21; Ipph=22; Ipps=23
N_STATE = 24

```

```

IxS=0; IyS=1; IzS=2; IpxS=3; IpyS=4; IpzS=5
IxE=6; IyE=7; IzE=8; IpxE=9; IpyE=10; IpzE=11
IxL=12; IyL=13; IzL=14; IpxL=15; IpyL=16; IpzL=17
Ith=18; Iph=19; Ips=20; Ipth=21; Ipph=22; Ipps=23
N_STATE = 24

```

Código completo

```

"""
=====

=
MODELO 1 – Hamiltoniano Sol-Tierra-Luna
Eclipse Solar Total · 12 agosto 2026
=====

=
"""

import numpy as np
import time
import sys
import os
from datetime import datetime, timedelta

sys.path.insert(0, os.path.dirname(__file__))
from benchmark import eclipse_error, print_benchmark, load_reference_data,
ECLIPSE_UTC

from constantes import (AU, DAY, G, mS, mE, mL, I_perp, I_z, dI,
                        GmS, GmE, GmL, theta_0, phi_0, psi_0,
                        ptheta_0, pphi_0, ppsi_0, C_YOSHIDA, D_YOSHIDA)

IxS=0; IyS=1; IzS=2; IpxS=3; IpyS=4; IpzS=5
IxE=6; IyE=7; IzE=8; IpxE=9; IpyE=10; IpzE=11
IxL=12; IyL=13; IzL=14; IpxL=15; IpyL=16; IpzL=17
Ith=18; Iph=19; Ips=20; Ipth=21; Ipph=22; Ipps=23
N_STATE = 24

def derivadas(state):

```

```

xS, yS, zS = state[IxS], state[IyS], state[IzS]
pxS, pyS, pzS = state[IpXS], state[IpYS], state[IpzS]
xE, yE, zE = state[IxE], state[IyE], state[IzE]
pxE, pyE, pZE = state[IpxE], state[IpYE], state[IpzE]
xL, yL, zL = state[IxL], state[IyL], state[IzL]
pxL, pyL, pzL = state[IpXL], state[IpYL], state[IpzL]
th, ph, ps = state[Ith], state[Iph], state[Ips]
pth, pph, pps = state[Ipth], state[IpPh], state[IpPs]

x_EL = xL-xE; y_EL = yL-yE; z_EL = zL-zE
x_ES = xS-xE; y_ES = yS-yE; z_ES = zS-zE
x_SL = xL-xS; y_SL = yL-yS; z_SL = zL-zS

r_EL2=x_EL**2+y_EL**2+z_EL**2; r_ES2=x_ES**2+y_ES**2+z_ES**2;
r_SL2=x_SL**2+y_SL**2+z_SL**2
r_EL=np.sqrt(r_EL2); r_ES=np.sqrt(r_ES2); r_SL=np.sqrt(r_SL2)
r_EL3=r_EL2*r_EL; r_ES3=r_ES2*r_ES; r_SL3=r_SL2*r_SL
r_EL5=r_EL3*r_EL2; r_ES5=r_ES3*r_ES2

sin_th=np.sin(th); cos_th=np.cos(th); sin_ph=np.sin(ph); cos_ph=np.cos(ph)
up_x=sin_th*sin_ph; up_y=-sin_th*cos_ph; up_z=cos_th
uL=x_EL*up_x+y_EL*up_y+z_EL*up_z; uS=x_ES*up_x+y_ES*up_y+z_ES*up_z
uθL=x_EL*cos_th*sin_ph-y_EL*cos_th*cos_ph-z_EL*sin_th
uφL=x_EL*sin_th*cos_ph+y_EL*sin_th*sin_ph
uθS=x_ES*cos_th*sin_ph-y_ES*cos_th*cos_ph-z_ES*sin_th
uφS=x_ES*sin_th*cos_ph+y_ES*sin_th*sin_ph
CL=3.0*GmL*dI/(2.0*r_EL5); CS=3.0*GmS*dI/(2.0*r_ES5)
fL=5.0*uL**2/r_EL2-1.0; fS=5.0*uS**2/r_ES2-1.0
sin_th_safe = 1e-10*np.sign(sin_th+1e-20) if abs(sin_th)<1e-10 else sin_th
pph_m=pph-pps*cos_th

dx_S=pxS/mS; dy_S=pyS/mS; dz_S=pzS/mS
dx_E=pxE/mE; dy_E=pyE/mE; dz_E=pzE/mE
dx_L=pxL/mL; dy_L=pyL/mL; dz_L=pzL/mL
dth=pth/I_perp; dph=pph_m/(I_perp*sin_th_safe**2)
dps=pps/I_z-pph_m*cos_th/(I_perp*sin_th_safe**2)

dpxS=(GmL*mS/r_SL3*x_SL-GmE*mS/r_ES3*(-x_ES)-CS*(fS*(-x_ES)-2.0*uS*up_x))
dpyS=(GmL*mS/r_SL3*y_SL-GmE*mS/r_ES3*(-y_ES)-CS*(fS*(-
y_ES)+2.0*uS*sin_th*cos_ph))
dpzS=(GmL*mS/r_SL3*z_SL-GmE*mS/r_ES3*(-z_ES)-CS*(fS*(-
z_ES)-2.0*uS*cos_th))
dpxE=(GmE*mL/r_EL3*x_EL+CL*(fL*x_EL-2.0*uL*up_x)+GmE*mS/r_ES3*x_ES+CS*
(fS*x_ES-2.0*uS*up_x))
dpyE=(GmE*mL/r_EL3*y_EL+CL*
(fL*y_EL+2.0*uL*sin_th*cos_ph)+GmE*mS/r_ES3*y_ES+CS*

```

```

(fS*y_ES+2.0*uS*sin_th*cos_ph))
    dpzE=(GmE*mL/r_EL3*z_EL+CL*(fL*z_EL-2.0*uL*cos_th)+GmE*mS/r_ES3*z_ES+CS*
(fS*z_ES-2.0*uS*cos_th))
    dpxE=(-GmL*mS/r_SL3*x_SL-GmE*mL/r_EL3*x_EL-CL*(fL*x_EL-2.0*uL*up_x))
    dpYL=(-GmL*mS/r_SL3*y_SL-GmE*mL/r_EL3*y_EL-CL*
(fL*y_EL+2.0*uL*sin_th*cos_ph))
    dpzL=(-GmL*mS/r_SL3*z_SL-GmE*mL/r_EL3*z_EL-CL*(fL*z_EL-2.0*uL*cos_th))
    dpth=(pph_m**2*cos_th/(I_perp*sin_th_safe**3)-
pph_m*pps/(I_perp*sin_th_safe)+2.0*CL*uL*uθL+2.0*CS*uS*uθS)
    dpph=2.0*CL*uL*uφL+2.0*CS*uS*uφS; dpps=0.0

    return
np.array([dx_S, dy_S, dz_S, dpxE, dpYL, dpzL, dth, dph, dps, dpth, dpph, dpps])

dx_L, dy_L, dz_L, dpXL, dpYL, dpzL, dth, dph, dps, dpth, dpph, dpps])

_C=C_YOSHIDA; _D=D_YOSHIDA
_IDX_Q=np.array([IxS, IyS, IzS, IxE, IyE, IzE, IxL, IyL, IzL, Ith, Iph, Ips])
_IDX_P=np.array([IpxS, IpyS, IpzS, IpxE, IpyE, IpzE, IpxL, IpyL, IpzL, Ipth, Ipph, Ipps])

def yoshida4(state, h):
    s=state.copy()
    for k in range(3):
        s[_IDX_Q]+=_C[k]*h*derivadas(s][_IDX_Q]
        ds=derivadas(s); s[_IDX_P]+=_D[k]*h*ds[_IDX_P]
    s[_IDX_Q]+=_C[3]*h*derivadas(s][_IDX_Q]
    return s

def energia(state):
    xS, yS, zS=state[IxS], state[IyS], state[IzS];
    pXS, pyS, pzS=state[IpxS], state[IpyS], state[IpzS]
    xE, yE, zE=state[IxE], state[IyE], state[IzE];
    pxE, pyE, pZE=state[IpxE], state[IpyE], state[IpzE]
    xL, yL, zL=state[IxL], state[IyL], state[IzL];
    pXL, pyL, pZL=state[IpxL], state[IpyL], state[IpzL]
    th, ph=state[Ith], state[Iph];
    pth, pph, pps=state[Ipth], state[Ipph], state[Ipps]
    T_tras=((pXS**2+pyS**2+pZS**2)/(2*mS)+(pXE**2+pyE**2+pZE**2)/(2*mE)+
(pXL**2+pyL**2+pZL**2)/(2*mL))
    sin_th=np.sin(th); cos_th=np.cos(th); sin_s=max(abs(sin_th), 1e-
10)*np.sign(sin_th+1e-20)
    pph_m=pph-pps*cos_th
    T_rot=pth**2/(2*I_perp)+pph_m**2/(2*I_perp*sin_s**2)+pps**2/(2*I_z)
    x_EL=xL-xE; y_EL=yL-yE; z_EL=zL-zE; x_ES=xS-xE; y_ES=yS-yE; z_ES=zS-zE; x_SL=xL-
xS; y_SL=yL-yS; z_SL=zL-zS
    r_EL=np.sqrt(x_EL**2+y_EL**2+z_EL**2);

```

```

r_ES=np.sqrt(x_ES**2+y_ES**2+z_ES**2); r_SL=np.sqrt(x_SL**2+y_SL**2+z_SL**2)
up_x=np.sin(th)*np.sin(ph); up_y=-np.sin(th)*np.cos(ph); up_z=np.cos(th)
uL=x_EL*up_x+y_EL*up_y+z_EL*up_z; uS=x_ES*up_x+y_ES*up_y+z_ES*up_z
V_SL=-GmL*mS/r_SL
V_EL=(-GmE*mL/r_EL-GmL*dI/(2*r_EL**3)*(3*uL**2/r_EL**2-1))
V_ES=(-GmE*mS/r_ES-GmS*dI/(2*r_ES**3)*(3*uS**2/r_ES**2-1))
return T_tras+T_rot+V_SL+V_EL+V_ES

def construir_estado_inicial(ci_dict):
    def pos_vel(nombre, masa):
        pos=np.array(ci_dict[nombre]["pos"]); vel=np.array(ci_dict[nombre]
["vel"]); return pos, masa*vel
    rS,pS=pos_vel("Sol",mS); rE,pE=pos_vel("Tierra",mE);
rL,pL=pos_vel("Luna",mL)
    state=np.zeros(N_STATE)
    state[IxS:IzS+1]=rS; state[IpxS:IpzS+1]=pS
    state[IxE:IzE+1]=rE; state[IpxE:IpzE+1]=pE
    state[IxL:IzL+1]=rL; state[IpxL:IpzL+1]=pL
    state[Ith]=theta_0; state[Iph]=phi_0; state[Ips]=psi_0
    state[Ipth]=ptheta_0; state[Ipvh]=pphi_0; state[Ipps]=ppsi_0
    return state

def buscar_eclipse(state, t_dias, h, t_inicio):
    sep_min=999.0; state_ecl=None; t_ecl_dias=t_dias
    while True:
        vES=state[IxS:IzS+1]-state[IxE:IzE+1]; vEL=state[IxL:IzL+1]-
state[IxE:IzE+1]
        cos_=np.clip(np.dot(vES,vEL)/(np.linalg.norm(vES)*np.linalg.norm(vEL)),-1,1)
        sep=np.degrees(np.arccos(cos_))
        if sep<sep_min: sep_min=sep; state_ecl=state.copy(); t_ecl_dias=t_dias
        state=yoshida4(state,h); t_dias+=h
        if sep_min<1.0 and sep>sep_min+0.1: break
        if t_dias>(datetime(2026,8,20)-t_inicio).days: break
    return state_ecl, t_inicio+timedelta(days=float(t_ecl_dias)), sep_min

def run(ci_dict=None, h=0.005, verbose=True):
    if ci_dict is None:
        print("AVISO: usando condiciones iniciales placeholder."); return
    T_INICIO=datetime(2026,1,1); DIAS=(datetime(2026,8,12)-T_INICIO).days
    state=construir_estado_inicial(ci_dict); E0=energia(state)
    if verbose:
        print(f"\n{'='*60}\n  MODELO 1 - Hamiltoniano Sol-Tierra-
Luna\n{'='*60}")
        print(f"  h={h}d={h*DAY:.0f}s | {DIAS} días | Eo={E0:.6e}\n")
        t0c=time.time(); t=0.0; N_p=int(DIAS/h); Nr=max(1,N_p//10); Em=0.0

```

```

for i in range(N_p):
    state=yoshida4(state,h); t+=h
    if verbose and i%Nr==0:
        E=energia(state); dE=abs((E-E0)/E0) if E0 else 0; Em=max(Em,dE)
        print(f" {(T_INICIO+timedelta(days=float(t))).strftime('%d %b %Y')} | ΔE/E={dE:.2e} | d_Luna={np.linalg.norm(state[IxL:IzL+1]-state[IxE:IzE+1])*AU/1e3:.0f} km")
    tlc=time.time()
    state_ecl,t_ecl,sep=buscar_eclipse(state,t,h/10,T_INICIO)
    pos_sol=state_ecl[IxS:IzS+1]; pos_tierra=state_ecl[IxE:IzE+1];
    pos_luna=state_ecl[IxL:IzL+1]
    ref=load_reference_data(); ref_ecl=ref.get(ECLIPSE.UTC.isoformat()) if ref
    else None
    m=eclipse_error(pos_sol,pos_tierra,pos_luna,t_ecl,ref_ecl)
    if verbose:
        print(f"\n CPU={t1c-t0c:.1f}s | ΔE/E_max={Em:.2e} | θ={sep:.4f} °")

        print(f" Máximo: {t_ecl.strftime('%d %b %Y %H:%M UTC')}")
        print(f" Eclipse total: {'SÍ ✓' if m['es_total'] else 'NO ✗'} |
Err_t={m['error_tiempo_min']} min")
    return {"nombre":"Modelo 1: H Sol-Tierra-Luna", "metricas":m, "tiempo_cpu_s":t1c-t0c,
            "descripcion":f"3 cuerpos + J2 + Euler, h={h}d", "_E_max_error":Em, "_t_ecl":t_ecl,
            "_sep_min_deg":sep, "_state_ecl":state_ecl}

if __name__=="__main__":
    import argparse
    p=argparse.ArgumentParser(); p.add_argument("--
h", type=float, default=0.005)
    p.add_argument("--rapido", action="store_true"); p.add_argument("--
benchmark", action="store_true")
    args=p.parse_args(); h=0.05 if args.rapido else args.h
    ref=load_reference_data(); ci=ref.get(datetime(2026,1,1).isoformat()) if
ref else None
    r=run(ci_dict=ci,h=h,verbose=True)
    if args.benchmark and r: print_benchmark([r])

```

References

Bibliografía usada:

- Mecánica Clásica - Goldstein

- Mecánica Clásica - Taylor
- GEOMETRIC NUMERICAL INTEGRATION STRUCTURE-PRESERVING ALGORITHMS FOR ORDINARY DIFFERENTIAL EQUATIONS SECOND EDITION - Hairer

Códigos

constantes.py

```
"""
CONSTANTES FÍSICAS Y ASTRONÓMICAS
Base para todos los modelos del Eclipse 2026.
"""

import numpy as np

# ——— CONSTANTES FUNDAMENTALES

AU = 1.495978707e11 # m
DAY = 86400.0 # s
G = 6.6743e-11 * DAY**2 / AU**3 # AU³ / (kg·día²)

# ——— CUERPOS Y MASAS

NOMBRES = ['Sol', 'Tierra', 'Luna', 'Mercurio', 'Venus', 'Marte',
            'Jupiter', 'Saturno', 'Urano', 'Neptuno']

MASAS = np.array([
    1.98892e30, # Sol
    5.97219e24, # Tierra
    7.34581e22, # Luna
    3.30200e23, # Mercurio
    4.86850e24, # Venus
    6.41712e23, # Marte
    1.89813e27, # Jupiter
    5.68319e26, # Saturno
    8.68103e25, # Urano
    1.02410e26, # Neptuno
])
N = len(MASAS)
GM = G * MASAS # AU³/día²

# Índices base
I_SOL = 0
I_TIE = 1
```

```

I_LUN = 2

# Masas específicas (legacy variable names para compatibilidad)
mS = MASAS[I_SOL]
mE = MASAS[I_TIE]
mL = MASAS[I_LUN]
GmS = GM[I_SOL]
GmE = GM[I_TIE]
GmL = GM[I_LUN]

# ——— MOMENTOS DE INERCIA (Tierra)


---


_m2AU2 = 1.0 / AU**2
I_perp = 8.0096e37 * _m2AU2
I_z     = 8.0358e37 * _m2AU2
dI      = I_z - I_perp

# ——— CONDICIONES INICIALES DE ROTACIÓN TERRESTRE


---


eps      = np.radians(23.4393)    # oblicuidad J2000
theta_0  = eps                    #  $\theta$  inicial
phi_0    = 0.0                    #  $\varphi$  inicial (longitud del nodo ascendente)
psi_0    = 0.0                    #  $\psi$  inicial (ángulo de rotación propia)

omega_rot = 2 * np.pi / 0.9972697 # rad/día (período sidéreo)
phi_dot_0 = 0.0                    # precesión despreciable
psi_dot_0 = omega_rot - phi_dot_0 * np.cos(theta_0)

# Momentos conjugados iniciales de la rotación
ptheta_0 = I_perp * 0.0            #  $\dot{\theta} = 0$  (eje polar estable)
ppsi_0    = I_z * psi_dot_0        #  $p\psi = I_z \cdot (\dot{\psi} + \dot{\varphi} \cos\theta)$ 
pphi_0    = I_perp * phi_dot_0 * np.sin(theta_0)**2 + ppsi_0 * np.cos(theta_0)

# ——— COEFICIENTES INTEGRADOR YOSHIDA O(4)


---


_w1 = 1.0 / (2.0 - 2.0**(1.0/3.0))
_w0 = -2.0**(1.0/3.0) * _w1
C_YOSHIDA = np.array([_w1/2.0, (_w0+_w1)/2.0, (_w0+_w1)/2.0, _w1/2.0])
D_YOSHIDA = np.array([_w1, _w0, _w1])

```

benchmark.py

```
"""
```

```
=
```

```
BENCHMARK – Proyecto Eclipse Solar Total 12 agosto 2026
```

```
=
```

Estructura del archivo:

1. `fetch_reference_data()` → descarga datos JPL Horizons (ejecutar una vez)
2. `load_reference_data()` → carga los datos guardados
3. `eclipse_error()` → calcula las 3 métricas de error para cualquier modelo
4. `print_benchmark()` → tabla comparativa de todos los modelos

Uso:

```
python benchmark.py --fetch      # descargar datos de referencia de JPL
python benchmark.py --check      # verificar datos descargados
python benchmark.py --demo       # mostrar tabla de ejemplo con un modelo
dummy
"""
```

```
import numpy as np
import json
import os
import argparse
from datetime import datetime, timedelta
```

```
# ——— CONSTANTES
```

```
AU          = 1.495978707e11 # metros
DAY          = 86400.0       # segundos
KM           = 1e3           # metros
ARCMIN       = 1/60.0        # grados
```

```
R_SOL       = 6.957e8 / AU
R_LUNA      = 1.7374e6 / AU
R_TIERRA    = 6.371e6 / AU
```

```
ECLIPSE.UTC = datetime(2026, 8, 12, 17, 47, 6)
```

```
FECHAS_CONTROL = [
    datetime(2026, 1, 1, 0, 0, 0),
    datetime(2026, 3, 1, 0, 0, 0),
    datetime(2026, 6, 1, 0, 0, 0),
    datetime(2026, 8, 1, 0, 0, 0),
    datetime(2026, 8, 12, 17, 47, 6),
```

```
]
```

```
REFERENCE_FILE = "Datos/jpl_reference_data.json"
```

```
# — 1. DESCARGA DE DATOS DE REFERENCIA
```

```
def fetch_reference_data():
    try:
        from astroquery.jplhorizons import Horizons
        print("Usando astroquery.jplhorizons")
        return _fetch_con_astroquery(Horizons)
    except ImportError:
        print("astroquery no disponible, usando API REST directa.")
    try:
        import requests
    except ImportError:
        print("ERROR: instala requests: pip install requests")
        return False
    return _fetch_con_requests(requests)

def _fetch_con_astroquery(Horizons):
    cuerpos = [("10", "Sol"), ("399", "Tierra"), ("301", "Luna"),
               ("199", "Mercurio"),
               ("299", "Venus"), ("499", "Marte"), ("599", "Jupiter"),
               ("699", "Saturno"),
               ("799", "Urano"), ("899", "Neptuno")]
    datos = {}
    for fecha in FECHAS_CONTROL:
        from astropy.time import Time
        t = Time(fecha.isoformat(), format='isot', scale='utc')
        print(f"\nFecha: {fecha.strftime('%Y-%b-%d %H:%M')}")
        datos[fecha.isoformat()] = {}
        for jpl_id, nombre in cuerpos:
            try:
                obj = Horizons(id=jpl_id, location="500@0", epochs=t.jd,
                               id_type="majorbody")
                vecs = obj.vectors(refplane="ecliptic")
                x, y, z = float(vecs["x"][0]), float(vecs["y"]
[0]), float(vecs["z"][0])
                vx, vy, vz = float(vecs["vx"][0]), float(vecs["vy"]
[0]), float(vecs["vz"][0])
                r = np.sqrt(x**2+y**2+z**2)
                datos[fecha.isoformat()][nombre] = {"pos": [x, y, z], "vel":
[vx, vy, vz], "r": float(r)}
```

```

        print(f" {nombre}: r={r:.6f} AU OK")
    except Exception as e:
        print(f" {nombre}: ERROR - {e}")
with open(REFERENCE_FILE, "w") as f: json.dump(datos, f, indent=2)
print(f"\n/ Datos guardados en {REFERENCE_FILE}")
return True

def _fetch_con_requests(requests):
    BASE_URL = "https://ssd.jpl.nasa.gov/api/horizons.api"
    cuerpos = [("10", "Sol"), ("399", "Tierra"), ("301", "Luna"),
                ("199", "Mercurio"),
                ("299", "Venus"), ("499", "Marte"), ("599", "Jupiter"),
                ("699", "Saturno"),
                ("799", "Urano"), ("899", "Neptuno")]
    datos = {}
    for fecha in FECHAS_CONTROL:
        fecha_str = fecha.strftime("%Y-%b-%d %H:%M")
        fecha_fin_str = (fecha+timedelta(minutes=2)).strftime("%Y-%b-%d
%H:%M")
        print(f"\nFecha: {fecha_str}")
        datos[fecha.isoformat()] = {}
        for jpl_id, nombre in cuerpos:
            params =
{"format": "text", "COMMAND": jpl_id, "OBJ_DATA": "NO", "MAKE_EPHEM": "YES",

"EPHEM_TYPE": "VECTORS", "CENTER": "500@0", "START_TIME": fecha_str,
                "STOP_TIME": fecha_fin_str, "STEP_SIZE": "1
m", "VEC_TABLE": "2",

"REF_PLANE": "ECLIPTIC", "REF_SYSTEM": "J2000", "OUT_UNITS": "AU-D",
                "VEC_LABELS": "YES", "CSV_FORMAT": "NO"}
            try:
                r = requests.get(BASE_URL, params=params, timeout=30)
                r.raise_for_status()
                rv = _parse_horizons_texto(r.text)
                if rv:
                    datos[fecha.isoformat()][nombre] = rv
                    print(f" {nombre}: r={rv['r']:.6f} AU OK")
                else:
                    print(f" {nombre}: ERROR al parsear")
            except Exception as e:
                print(f" {nombre}: EXCEPCION - {e}")
with open(REFERENCE_FILE, "w") as f: json.dump(datos, f, indent=2)
print(f"\n/ Datos guardados en {REFERENCE_FILE}")
return True

```

```

def _parse_horizons_texto(text):
    import re
    pos = None; vel = None
    for line in text.split("\n"):
        line_s = line.strip()
        if not line_s: continue
        if pos is None and re.search(r'\bX\s*=', line_s) and
re.search(r'\bY\s*=', line_s):
            nums = re.findall(r'[XYZ]\s*=\s*([+-]?\d+\.\d+[Ee][+-]?\d+)', line_s)
            if len(nums)>=3: pos=
[float(nums[0]),float(nums[1]),float(nums[2])]
            if vel is None and re.search(r'\bVX\s*=', line_s) and
re.search(r'\bVY\s*=', line_s):
                nums = re.findall(r'V[XYZ]\s*=\s*([+-]?\d+\.\d+[Ee][+-]?\d+)', line_s)
                if len(nums)>=3: vel=
[float(nums[0]),float(nums[1]),float(nums[2])]
                if pos is not None and vel is not None: break
            if pos is not None and vel is not None:
                return {"pos":pos, "vel":vel, "r":float(np.linalg.norm(pos))}
    return None

```

— 2. CARGA DE DATOS DE REFERENCIA

```

def load_reference_data():
    if not os.path.exists(REFERENCE_FILE):
        print(f"ERROR: no existe {REFERENCE_FILE}. Ejecuta: python
benchmark.py --fetch")
        return None
    with open(REFERENCE_FILE) as f: datos = json.load(f)
    print(f"✓ Datos de referencia cargados: {len(datos)} fechas ×
{len(list(datos.values())[0])} cuerpos")
    return datos

```

— 3. CÁLCULO DE ERROR

```

def eclipse_error(pos_sol, pos_tierra, pos_luna, t_eclipse_pred=None,
referencia=None):
    pos_sol=np.array(pos_sol); pos_tierra=np.array(pos_tierra);

```

```

pos_luna=np.array(pos_luna)
v_tl=pos_luna-pos_tierra; v_ts=pos_sol-pos_tierra
d_luna=np.linalg.norm(v_tl); d_sol=np.linalg.norm(v_ts)
cos_sep=np.clip(np.dot(v_ts,v_tl)/(d_sol*d_luna),-1,1)
sep_grados=np.degrees(np.arccos(cos_sep)); sep_arcmin=sep_grados*60.0
r_ang_sol=np.degrees(np.arcsin(np.clip(R_SOL/d_sol,0,1)))*60
r_ang_luna=np.degrees(np.arcsin(np.clip(R_LUNA/d_luna,0,1)))*60
fase_magnitud=r_ang_luna/r_ang_sol
es_total=(fase_magnitud>1.0) and (sep_arcmin<r_ang_sol)
if referencia is not None:
    ref_luna=np.array(referencia["Luna"]["pos"]);
ref_tierra=np.array(referencia["Tierra"]["pos"])
    error_luna_km=np.linalg.norm((pos_luna-pos_tierra)-(ref_luna-
ref_tierra))*AU/KM
    ref_sol=np.array(referencia["Sol"]["pos"]); v_rl=ref_luna-ref_tierra;
v_rs=ref_sol-ref_tierra

cos_ref=np.clip(np.dot(v_rs,v_rl)/(np.linalg.norm(v_rs)*np.linalg.norm(v_rl)),
-1,1)
    error_ang_arcmin=abs(sep_arcmin-np.degrees(np.arccos(cos_ref))*60.0)
else:
    error_luna_km=None; error_ang_arcmin=None
    error_tiempo_min=abs((t_eclipse_pred-ECLIPSE.UTC).total_seconds()/60.0) if
t_eclipse_pred else None
    return
{"sep_sol_luna_arcmin":round(sep_arcmin,4), "r_ang_sol_arcmin":round(r_ang_sol,
4),

"r_ang_luna_arcmin":round(r_ang_luna,4), "fase_magnitud":round(fase_magnitud,5)
,
    "es_total":es_total, "d_luna_km":round(d_luna*AU/KM),
    "error_pos_luna_km":round(error_luna_km) if error_luna_km is not
None else None,
    "error_ang_arcmin":round(error_ang_arcmin,3) if error_ang_arcmin
is not None else None,
    "error_tiempo_min":round(error_tiempo_min,1) if error_tiempo_min
is not None else None}

```

— 4. TABLA COMPARATIVA

```

def print_benchmark(resultados):
    sep="—" * 95
    print("\n\n┌" + "=" * 93 + "┐")
    print("|| BENCHMARK - Eclipse Solar Total · 12 agosto 2026 · Error vs JPL

```

```

Horizons DE441"+" "*11+"||")
    print("L"+"="*93+"J")
    print(f"\n{'Modelo':<30} {'d_Luna':>10} {'Err_pos':>12} {'Err_ang':>12}
{'Err_t':>8} {'Total?':>7} {'CPU':>6}")
    print(f"{'':<30} {'km':>10} {'km':>12} {'arcmin':>12} {'min':>8}
{'':>7} {'s':>6}")
    print(sep)
    for r in resultados:
        m=r["metricas"]
        ep=f"{m['error_pos_luna_km']:.0f}" if m["error_pos_luna_km"] is not
None else "-"
        ea=f"{m['error_ang_arcmin']:.3f}" if m["error_ang_arcmin"] is not
None else "-"
        et=f"{m['error_tiempo_min']:.1f}" if m["error_tiempo_min"] is not
None else "-"
        tt="/ TOTAL" if m["es_total"] else "X anular"
        print(f"{r['nombre']:<30} {m['d_luna_km']:>10,} {ep:>12} {ea:>12}
{et:>8} {tt:>7} {r.get('tiempo_cpu_s',0):>6.1f}")
    print(sep)

if __name__=="__main__":
    parser=argparse.ArgumentParser()
    parser.add_argument("--fetch",action="store_true"); parser.add_argument("--
demo",action="store_true")
    parser.add_argument("--all",action="store_true"); args=parser.parse_args()
    if args.fetch: fetch_reference_data()
    elif args.demo: demo_con_v10()
    elif args.all: comparar_todos()
    else: parser.print_help()

```

modelo2_ncuerpos.py

```

"""
MODELO 2 – Hamiltoniano N-cuerpos completo
Eclipse Solar Total · 12 agosto 2026

Extiende Modelo 1 añadiendo los 7 planetas restantes como masas puntuales.
Cuerpos (índices): 0=Sol 1=Tierra 2=Luna 3=Mercurio 4=Venus 5=Marte
                   6=Jupiter 7=Saturno 8=Urano 9=Neptuno
Estado: 66 variables
"""

import numpy as np

```



```

import time
import sys
import os
from datetime import datetime, timedelta

sys.path.insert(0, os.path.dirname(__file__))
from benchmark import eclipse_error, print_benchmark, load_reference_data,
ECLIPSE_UTC
from constantes import (AU, DAY, G, NOMBRES, MASAS, N, GM,
                        I_SOL, I_TIE, I_LUN, I_perp, I_z, dI,
                        theta_0, phi_0, psi_0, ptheta_0, pphi_0, ppsi_0,
                        C_YOSHIDA, D_YOSHIDA)

def ir(i): return 6*i
def ip(i): return 6*i+3

N_TRAS=6*N; N_STATE=N_TRAS+6
ITH=60; IPH=61; IPS=62; IPTH=63; IPPH=64; IPPS=65

def derivadas(s):
    ds=np.zeros(N_STATE)
    r=np.array([s[ir(i):ir(i)+3] for i in range(N)])
    p=np.array([s[ip(i):ip(i)+3] for i in range(N)])
    th=s[ITH]; ph=s[IPH]; pth=s[IPTH]; pph=s[IPPH]; pps=s[IPPS]
    sin_th=np.sin(th); cos_th=np.cos(th); sin_ph=np.sin(ph); cos_ph=np.cos(ph)
    sin_th_s=sin_th if abs(sin_th)>1e-10 else np.sign(sin_th+1e-20)*1e-10
    pph_m=pph-pps*cos_th
    up=np.array([sin_th*sin_ph, -sin_th*cos_ph, cos_th])
    for i in range(N): ds[ir(i):ir(i)+3]=p[i]/MASAS[i]
    ds[ITH]=pth/I_perp; ds[IPH]=pph_m/(I_perp*sin_th_s**2)
    ds[IPS]=pps/I_z-pph_m*cos_th/(I_perp*sin_th_s**2)
    for i in range(N):
        for j in range(i+1,N):
            dr=r[j]-r[i]; d2=np.dot(dr,dr); d3=d2*np.sqrt(d2); f=dr/d3
            ds[ip(i):ip(i)+3]+=MASAS[i]*GM[j]*f; ds[ip(j):ip(j)+3]-
=MASAS[j]*GM[i]*f
    for ext in [I_LUN,I_SOL]:
        dr=r[ext]-r[I_TIE]; d2=np.dot(dr,dr); d=np.sqrt(d2); d5=d2*d2*d
        u=np.dot(dr,up); C=3.0*GM[ext]*dI/(2.0*d5); fJ2=5.0*u**2/d2-1.0
        g=C*(fJ2*dr-2.0*u*up)
        ds[ip(ext):ip(ext)+3]-=g; ds[ip(I_TIE):ip(I_TIE)+3]+=g
    r_EL=r[I_LUN]-r[I_TIE]; r_ES=r[I_SOL]-r[I_TIE]
    d_EL2=np.dot(r_EL,r_EL); d_EL=np.sqrt(d_EL2); d_EL5=d_EL2**2*d_EL
    d_ES2=np.dot(r_ES,r_ES); d_ES=np.sqrt(d_ES2); d_ES5=d_ES2**2*d_ES
    uL=np.dot(r_EL,up); uS=np.dot(r_ES,up)
    uTL=r_EL[0]*cos_th*sin_ph-r_EL[1]*cos_th*cos_ph-r_EL[2]*sin_th

```

```

uFL=r_EL[0]*sin_th*cos_ph+r_EL[1]*sin_th*sin_ph
uTS=r_ES[0]*cos_th*sin_ph-r_ES[1]*cos_th*cos_ph-r_ES[2]*sin_th
uFS=r_ES[0]*sin_th*cos_ph+r_ES[1]*sin_th*sin_ph
CL=3.0*GM[I_LUN]*dI/(2.0*d_EL5); CS=3.0*GM[I_SOL]*dI/(2.0*d_ES5)
ds[IPTH]=(pph_m**2*cos_th/(I_perp*sin_th_s**3)-
pph_m*pps/(I_perp*sin_th_s)+2.0*CL*uL*uTL+2.0*CS*uS*uTS)
ds[IPPH]=2.0*CL*uL*uFL+2.0*CS*uS*uFS; ds[IPPS]=0.0
return ds

_C=C_YOSHIDA; _D=D_YOSHIDA
_IQ=np.array([ir(i)+k for i in range(N) for k in range(3)]+[ITH,IPH,IPS])
_IP=np.array([ip(i)+k for i in range(N) for k in range(3)]+[IPTH,IPPH,IPPS])

def yoshida4(s,h):
    s=s.copy()
    for k in range(3):
        ds=derivadas(s); s[_IQ]+=_C[k]*h*ds[_IQ]
        ds=derivadas(s); s[_IP]+=_D[k]*h*ds[_IP]
    ds=derivadas(s); s[_IQ]+=_C[3]*h*ds[_IQ]
    return s

def energia(s):
    r=np.array([s[ir(i):ir(i)+3] for i in range(N)]);
    p=np.array([s[ip(i):ip(i)+3] for i in range(N)])
    th=s[ITH]; ph=s[IPH]; pth=s[IPTH]; pph=s[IPPH]; pps=s[IPPS]
    sin_th=np.sin(th); cos_th=np.cos(th); sin_s=max(abs(sin_th),1e-
10)*np.sign(sin_th+1e-20)
    pph_m=pph-pps*cos_th; up=np.array([sin_th*np.sin(ph),-
sin_th*np.cos(ph),cos_th])
    T=sum(np.dot(p[i],p[i])/(2*MASAS[i]) for i in range(N))
    T+=pth**2/(2*I_perp)+pph_m**2/(2*I_perp*sin_s**2)+pps**2/(2*I_z)
    V=0.0
    for i in range(N):
        for j in range(i+1,N): V-=GM[i]*MASAS[j]/np.linalg.norm(r[j]-r[i])
    for ext in [I_LUN,I_SOL]:
        dr=r[ext]-r[I_TIE]; d=np.linalg.norm(dr); u=np.dot(dr,up)
        V-=GM[ext]*dI/(2*d**3)*(3*u**2/d**2-1)
    return T+V

NOMBRE_IDX=
{'Sol':0,'Tierra':1,'Luna':2,'Mercurio':3,'Venus':4,'Marte':5,'Jupiter':6,'Sat
urno':7,'Urano':8,'Neptuno':9}

def construir_estado(ci_dict):
    s=np.zeros(N_STATE)
    for nombre,idx in NOMBRE_IDX.items():

```

```

        if nombre not in ci_dict: continue
        s[ir(idx):ir(idx)+3]=np.array(ci_dict[nombre]['pos'])
        s[ip(idx):ip(idx)+3]=MASAS[idx]*np.array(ci_dict[nombre]['vel'])
s[ITH]=theta_0; s[IPH]=phi_0; s[IPS]=psi_0
s[IPTH]=ptheta_0; s[IPPH]=pphi_0; s[IPPS]=ppsi_0
return s

def buscar_eclipse(s,t_dias,h,t0):
    sep_min=999.0; s_ecl=None; t_ecl=t_dias
    while True:
        vES=s[ir(I_SOL):ir(I_SOL)+3]-s[ir(I_TIE):ir(I_TIE)+3]
        vEL=s[ir(I_LUN):ir(I_LUN)+3]-s[ir(I_TIE):ir(I_TIE)+3]

c=np.clip(np.dot(vES,vEL)/(np.linalg.norm(vES)*np.linalg.norm(vEL)),-1,1)
        sep=np.degrees(np.arccos(c))
        if sep<sep_min: sep_min=sep; s_ecl=s.copy(); t_ecl=t_dias
        s=yoshida4(s,h); t_dias+=h
        if sep_min<1.0 and sep>sep_min+0.1: break
        if t_dias>(datetime(2026,8,20)-t0).days: break
    return s_ecl, t0+timedelta(days=float(t_ecl)), sep_min

def run(h=0.005, verbose=True):
    T0=datetime(2026,1,1); DIAS=223
    ref=load_reference_data(); ci=ref.get(T0.isoformat()) if ref else None
    if ci is None: print("ERROR: ejecuta benchmark.py --fetch primero");
return None
    s=construir_estado(ci); E0=energia(s)
    if verbose:
        print(f"\n{'='*60}\n  MODELO 2 - N-cuerpos (10 cuerpos + J2 +
Euler)\n{'='*60}")
        print(f"  h={h}d={h*DAY:.0f}s | {DIAS} días | Eo={E0:.6e} |
{N_STATE} vars\n")
    t0c=time.time(); t=0.0; N_p=int(DIAS/h); Nr=max(1,N_p//10); Em=0.0
    for i in range(N_p):
        s=yoshida4(s,h); t+=h
        if verbose and i%Nr==0:
            E=energia(s); dE=abs((E-E0)/E0) if E0 else 0; Em=max(Em,dE)
            d=np.linalg.norm(s[ir(I_LUN):ir(I_LUN)+3]-
s[ir(I_TIE):ir(I_TIE)+3])*AU/1e3
            print(f"  {(T0+timedelta(days=float(t))).strftime('%d %b %Y')} |
ΔE/E={dE:.2e} | d_Luna={d:.0f} km")
    t1c=time.time()
    s_ecl,t_ecl,sep=buscar_eclipse(s,t,h/10,T0)
    ps=s_ecl[ir(I_SOL):ir(I_SOL)+3]; pt=s_ecl[ir(I_TIE):ir(I_TIE)+3];
pl=s_ecl[ir(I_LUN):ir(I_LUN)+3]
    ref_ecl=ref.get(ECLIPSE.UTC.isoformat()) if ref else None

```

```

m=eclipse_error(ps,pt,pl,t_ecl,ref_ecl)
if verbose:
    print(f"\n CPU={t1c-t0c:.1f}s | ΔE/E_max={Em:.2e} | θ={sep:.4f}
o")

    print(f" Máximo: {t_ecl.strftime('%d %b %Y %H:%M UTC')}")
    print(f" d_Luna={m['d_luna_km']:,} km | Total={'SÍ ✓' if
m['es_total'] else 'NO ✗'}")
    return {"nombre":"Modelo 2: H N-cuerpos","metricas":m,"tiempo_cpu_s":t1c-
t0c,

            "descripcion":f"10 cuerpos + J2 + Euler, h={h}d"}

if __name__=="__main__":
    import argparse
    p=argparse.ArgumentParser(); p.add_argument("--
h",type=float,default=0.005)
    p.add_argument("--rapido",action="store_true"); args=p.parse_args()
    h=0.05 if args.rapido else args.h
    run(h=h,verbose=True)

```

modelo3_relativista.py

```

"""
MODELO 3 – N-cuerpos + correcciones relativistas EIH

=====

Añade las correcciones post-newtonianas de Einstein-Infeld-Hoffmann (EIH).
Corrección EIH / aceleración newtoniana  $\sim v^2/c^2 \sim 1e-8$ 
Referencia: Soffel et al. 2003, AJ 126, 2687
"""

import numpy as np
import time
import sys
import os
from datetime import datetime, timedelta

sys.path.insert(0, os.path.dirname(__file__))
from benchmark import eclipse_error, print_benchmark, load_reference_data,
ECLIPSE_UTC
from constantes import (AU, DAY, MASAS, GM, N, I_SOL, I_TIE, I_LUN,
                        I_perp, I_z, dI, C_YOSHIDA as _C, D_YOSHIDA as _D)
from modelo2_ncuerpos import (construir_estado, ir, ip, N_STATE,

```

ITH, IPH, IPS, IPTH, IPPH, IPPS, _IQ, _IP)

C_LUZ = 299792.458e3 / (AU / DAY)

C2 = C_LUZ**2

```
def acels_newton(r):
    a=np.zeros((N,3))
    for i in range(N):
        for j in range(i+1,N):
            dr=r[j]-r[i]; d2=np.dot(dr,dr); d3=d2*np.sqrt(d2); f=dr/d3
            a[i]+=GM[j]*f; a[j]-=GM[i]*f
    return a

def correccion_eih(r,v,a_newton):
    da=np.zeros((N,3))
    Phi=np.zeros(N)
    for i in range(N):
        for j in range(N):
            if i!=j: Phi[i]+=GM[j]/np.linalg.norm(r[j]-r[i])
    for i in range(N):
        vi2=np.dot(v[i],v[i])
        for j in range(N):
            if i==j: continue
            dr=r[j]-r[i]; d=np.linalg.norm(dr); d2=d*d; nij=dr/d
            vj2=np.dot(v[j],v[j]); vivj=np.dot(v[i],v[j])
            nijvj=np.dot(nij,v[j]); nijvi=np.dot(nij,v[i]); dv=v[i]-v[j]
            A=(-vi2-2.0*vj2+4.0*vivj+1.5*nijvj**2+4.0*Phi[i]-Phi[j])
            B=4.0*nijvi-3.0*nijvj
            da[i]+=(GM[j]/(C2*d2))*(A*nij+B*dv)
        for j in range(N):
            if i==j: continue
            da[i]+=(3.5/C2)*(GM[j]/np.linalg.norm(r[j]-r[i]))*a_newton[j]
    return da

def derivadas_eih(s):
    ds=np.zeros(N_STATE)
    r=np.array([s[ir(i):ir(i)+3] for i in range(N)])
    p=np.array([s[ip(i):ip(i)+3] for i in range(N)])
    v=np.array([p[i]/MASAS[i] for i in range(N)])
    for i in range(N): ds[ir(i):ir(i)+3]=v[i]
    a_n=acels_newton(r); da_eih=correccion_eih(r,v,a_n)
    for i in range(N): ds[ip(i):ip(i)+3]=MASAS[i]*(a_n[i]+da_eih[i])
    return ds

def yoshida4(s,h):
    s=s.copy()
```

```

for k in range(3):
    ds=derivadas_eih(s); s[_IQ]+=_C[k]*h*ds[_IQ]
    ds=derivadas_eih(s); s[_IP]+=_D[k]*h*ds[_IP]
ds=derivadas_eih(s); s[_IQ]+=_C[3]*h*ds[_IQ]
return s

def energia(s):
    r=np.array([s[ir(i):ir(i)+3] for i in range(N)]);
    p=np.array([s[ip(i):ip(i)+3] for i in range(N)])
    T=sum(np.dot(p[i],p[i])/(2*MASAS[i]) for i in range(N))
    V=0.0
    for i in range(N):
        for j in range(i+1,N): V-=GM[i]*MASAS[j]/np.linalg.norm(r[j]-r[i])
    return T+V

def buscar_eclipse(s,t,h,t0):
    sep_min=999.0; s_ecl=None; t_ecl=t
    while True:
        vES=s[ir(I_SOL):ir(I_SOL)+3]-s[ir(I_TIE):ir(I_TIE)+3]
        vEL=s[ir(I_LUN):ir(I_LUN)+3]-s[ir(I_TIE):ir(I_TIE)+3]

        c=np.clip(np.dot(vES,vEL)/(np.linalg.norm(vES)*np.linalg.norm(vEL)),-1,1)
        sep=np.degrees(np.arccos(c))
        if sep<sep_min: sep_min=sep; s_ecl=s.copy(); t_ecl=t
        s=yoshida4(s,h); t+=h
        if sep_min<1.0 and sep>sep_min+0.1: break
        if t>(datetime(2026,8,20)-t0).days: break
    return s_ecl, t0+timedelta(days=float(t_ecl)), sep_min

def run(h=0.005, verbose=True):
    T0=datetime(2026,1,1); DIAS=223
    ref=load_reference_data(); ci=ref.get(T0.isoformat()) if ref else None
    if ci is None: print("ERROR: ejecuta benchmark.py --fetch"); return None
    s=construir_estado(ci); s[ITH:IPPS+1]=0.0; E0=energia(s)
    r0=np.array([s[ir(i):ir(i)+3] for i in range(N)]);
    p0=np.array([s[ip(i):ip(i)+3] for i in range(N)])
    v0=np.array([p0[i]/MASAS[i] for i in range(N)])
    a_n0=acels_newton(r0); da0=correccion_eih(r0,v0,a_n0)
    ratio_t=np.linalg.norm(da0[I_TIE])/np.linalg.norm(a_n0[I_TIE])
    ratio_l=np.linalg.norm(da0[I_LUN])/np.linalg.norm(a_n0[I_LUN])
    if verbose:
        print(f"\n{'='*60}\n  MODELO 3 - N-cuerpos + correcciones EIH
relativistas\n{'='*60}")
        print(f"  h={h}d |  c_luz={C_LUZ:.4f} AU/día |  EIH/Newton: Tierra=
{ratio_t:.2e}  Luna={ratio_l:.2e}\n")
    t0c=time.time(); t=0.0; N_p=int(DIAS/h); Nr=max(1,N_p//10); Em=0.0

```

```

for i in range(N_p):
    s=yoshida4(s,h); t+=h
    if verbose and i%Nr==0:
        E=energia(s); dE=abs((E-E0)/E0); Em=max(Em,dE)
        print(f"  {(T0+timedelta(days=float(t))).strftime('%d %b %Y')} |
ΔE/E={dE:.2e} | d_Luna={np.linalg.norm(s[ir(I_LUN):ir(I_LUN)+3]-
s[ir(I_TIE):ir(I_TIE)+3])*AU/1e3:.0f} km")
        tlc=time.time()
        s_ecl,t_ecl,sep=buscar_eclipse(s,t,h/10,T0)
        ps=s_ecl[ir(I_SOL):ir(I_SOL)+3]; pt=s_ecl[ir(I_TIE):ir(I_TIE)+3];
        pl=s_ecl[ir(I_LUN):ir(I_LUN)+3]
        ref_ecl=ref.get(ECLIPSE.UTC.isoformat()) if ref else None
        m=eclipse_error(ps,pt,pl,t_ecl,ref_ecl)
        if verbose:
            print(f"\n  CPU={t1c-t0c:.1f}s | ΔE/E_max={Em:.2e} | θ={sep:.4f}
o")

            print(f"  Máximo: {t_ecl.strftime('%d %b %Y %H:%M UTC')}")
            print(f"  d_Luna={m['d_luna_km'],} km | Total={'SÍ ✓' if
m['es_total'] else 'NO ✗'}")
            return {"nombre":"Modelo 3: N-cuerpos +
EIH","metricas":m,"tiempo_cpu_s":t1c-t0c,
                    "descripcion":f"10 cuerpos + EIH, h=
{h}d","_pos_sol":ps,"_pos_tierra":pt,

"_pos_luna":pl,"_t_ecl":t_ecl,"_ratio_eih_tierra":ratio_t,"_ratio_eih_luna":ra
tio_l}

if __name__=="__main__":
    import argparse
    p=argparse.ArgumentParser(); p.add_argument("--
h",type=float,default=0.005)
    p.add_argument("--rapido",action="store_true"); args=p.parse_args()
    run(h=0.05 if args.rapido else args.h, verbose=True)

```

modelo4_besseliano.py

```

"""
MODELO 4 – Método Geométrico Riguroso (Precisión NASA/ESA)
Eclipse Solar Total · 12 agosto 2026

Híbrido: integración física con Modelo 3 + proyección de sombra
con elementos besselianos y elipsoide WGS84.
"""

```

```

import numpy as np
import time
import sys
import os
from datetime import datetime, timedelta

sys.path.insert(0,
os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
from benchmark import eclipse_error, print_benchmark, load_reference_data,
ECLIPSE.UTC

AU = 1.495978707e11
R_SOL = 6.957e8; R_LUNA = 1.7374e6
EPSILON = np.radians(23.4392911)
WGS84_a = 6378137.0; WGS84_f = 1.0/298.257223563
WGS84_b = WGS84_a*(1.0-WGS84_f)
DELTA_T = 69.0

def ecliptic_to_equatorial(vec_ecl):
    x,y,z = vec_ecl
    cos_e=np.cos(EPSILON); sin_e=np.sin(EPSILON)
    return np.array([x, y*cos_e-z*sin_e, y*sin_e+z*cos_e])

def gmst_angle(dt_utc):
    J2000=datetime(2000,1,1,12,0,0)
    d=(dt_utc-J2000).total_seconds()/86400.0
    gmst_deg=((18.697374558+24.06570982441908*d)*15.0)%360.0
    return np.radians(gmst_deg)

def calcular_elementos_besselianos(rS_eq,rE_eq,rL_eq,utc_time):
    R_S=rS_eq-rE_eq; R_L=rL_eq-rE_eq
    G_vec=R_L-R_S; dist_G=np.linalg.norm(G_vec); k_hat=G_vec/dist_G
    d_rad=np.arcsin(k_hat[2]); a_rad=np.arctan2(k_hat[1],k_hat[0])
    i_hat=np.array([-np.sin(a_rad),np.cos(a_rad),0.0]);
    j_hat=np.cross(k_hat,i_hat)
    x=np.dot(R_L,i_hat); y=np.dot(R_L,j_hat)
    gmst=gmst_angle(utc_time); mu=(gmst-a_rad)%(2*np.pi)
    sin_f1=(R_SOL+R_LUNA)/dist_G; sin_f2=(R_SOL-R_LUNA)/dist_G
    z_L=np.dot(R_L,k_hat)
    l1=z_L*np.tan(np.arcsin(sin_f1))+R_LUNA/np.cos(np.arcsin(sin_f1))
    l2=z_L*np.tan(np.arcsin(sin_f2))-R_LUNA/np.cos(np.arcsin(sin_f2))
    return
    {"x":x,"y":y,"d":d_rad,"mu":mu,"l1":l1,"l2":l2,"a":a_rad,"i":i_hat,"j":j_hat,"
k":k_hat}

```



```

def interseccion_wgs84(elementos):
    x=elementos["x"]; y=elementos["y"]; d_rad=elementos["d"];
    mu_rad=elementos["mu"]

    lat_rad=np.arcsin(np.sin(d_rad)*np.cos(y/WGS84_a)+np.cos(d_rad)*np.sin(y/WGS84_a))
    lon_rad=(-mu_rad+np.arctan2(x,WGS84_a*np.cos(lat_rad)))
    lon_rad=(lon_rad+np.pi)%(2*np.pi)-np.pi
    return np.degrees(lat_rad), np.degrees(lon_rad)

import modelo3_relativista

def run(h=0.05, verbose=True):
    if verbose:
        print(f"\n{'='*60}\n  MODELO 4 - Híbrido: Física Simulada + Geometría NASA\n{'='*60}")
        print(f" Paso 1. Simulando el Sistema Solar (N-Cuerpos + EIH)...")
    t0_cpu=time.time()
    res3=modelo3_relativista.run(h=h,verbose=False)
    if res3 is None: print("Modelo 4: Falló la ejecución del Modelo 3.");
    return None
    pos_sol_ecl=res3["_pos_sol"]; pos_tie_ecl=res3["_pos_tierra"]
    pos_lun_ecl=res3["_pos_luna"]; t_ecl=res3["_t_ecl"]
    if verbose:
        print(f" Máximo simulado: {t_ecl.strftime('%d %b %Y %H:%M UTC')}")
        print(f" Paso 2. Aplicando Geometría Besseliana y Proyección WGS84...")
    rS_eq=ecliptic_to_equatorial(pos_sol_ecl)*AU
    rE_eq=ecliptic_to_equatorial(pos_tie_ecl)*AU
    rL_eq=ecliptic_to_equatorial(pos_lun_ecl)*AU
    elem=calcular_elementos_besselianos(rS_eq,rE_eq,rL_eq,t_ecl)
    lat_deg,lon_deg=interseccion_wgs84(elem)
    t1_cpu=time.time()
    if verbose:
        print(f" Declinación sombra (d): {np.degrees(elem['d']):.4f}°")
        print(f" Ángulo Horario Greenwich (μ): {np.degrees(elem['mu']):.4f}°")

    print(f" Coord X, Y del cono: {elem['x']/1000:,.1f} km, {elem['y']/1000:,.1f} km")
    print(f" Radio Penumbra (l1): {elem['l1']/1000:,.1f} km | Umbra (l2): {elem['l2']/1000:,.1f} km")
    print(f" Latitud: {lat_deg:.4f}° N | Longitud: {lon_deg:.4f}°")
    ref_datos=load_reference_data()
    ref_ecl=ref_datos.get(ECLIPSE.UTC.isoformat()) if ref_datos else None
    metricas=eclipse_error(pos_sol_ecl,pos_tie_ecl,pos_lun_ecl,t_ecl,ref_ecl)
    return {"nombre":"Modelo 4: Híbrido Físico-

```

```
Geométrico", "metricas": metricas,
    "tiempo_cpu_s": t1_cpu - t0_cpu, "descripcion": "Simulado (Mod3) +
    Bessel / WGS84"}

if __name__ == "__main__":
    r4 = run(verbose=True)
    if r4: print_benchmark([r4])
```

Explicación por apartados

constantes.py

Constantes fundamentales

```
AU = 1.495978707e11 # m
DAY = 86400.0 # s
G = 6.6743e-11 * DAY**2 / AU**3
```

Masas y nombres de los cuerpos

```
NOMBRES = ['Sol', 'Tierra', 'Luna', 'Mercurio', 'Venus', 'Marte',
            'Jupiter', 'Saturno', 'Urano', 'Neptuno']

MASAS = np.array([1.98892e30, 5.97219e24, 7.34581e22, 3.30200e23,
                  4.86850e24, 6.41712e23, 1.89813e27, 5.68319e26,
                  8.68103e25, 1.02410e26])

N = len(MASAS)
GM = G * MASAS
```

Momentos de inercia de la Tierra

```
_m2AU2 = 1.0 / AU**2
I_perp = 8.0096e37 * _m2AU2
I_z = 8.0358e37 * _m2AU2
dI = I_z - I_perp
```

Condiciones iniciales de rotación terrestre

```
eps = np.radians(23.4393)
theta_0 = eps
```

```

phi_0      = 0.0
psi_0      = 0.0
omega_rot  = 2 * np.pi / 0.9972697
ptheta_0   = I_perp * 0.0
ppsi_0     = I_z * psi_dot_0
pphi_0     = I_perp * phi_dot_0 * np.sin(theta_0)**2 + ppsi_0 * np.cos(theta_0)

```

Coeficientes de Yoshida O(4)

```

_w1 = 1.0 / (2.0 - 2.0**(1.0/3.0))
_w0 = -2.0**(1.0/3.0) * _w1
C_YOSHIDA = np.array([_w1/2.0, (_w0+_w1)/2.0, (_w0+_w1)/2.0, _w1/2.0])
D_YOSHIDA = np.array([_w1, _w0, _w1])

```

benchmark.py

Constantes y fecha del eclipse

```

AU = 1.495978707e11; DAY = 86400.0; KM = 1e3
R_SOL = 6.957e8 / AU; R_LUNA = 1.7374e6 / AU; R_TIERRA = 6.371e6 / AU
ECLIPSE.UTC = datetime(2026, 8, 12, 17, 47, 6)
REFERENCE_FILE = "Datos/jpl_reference_data.json"

```

Descarga de datos JPL (fetch_reference_data)

```

def fetch_reference_data():
    try:
        from astroquery.jplhorizons import Horizons
        return _fetch_con_astroquery(Horizons)
    except ImportError:
        pass
    import requests
    return _fetch_con_requests(requests)

```

Parser del formato de texto de Horizons

```

def _parse_horizons_texto(text):
    import re
    pos = None; vel = None
    for line in text.split("\n"):

```

```

line_s = line.strip()
if pos is None and re.search(r'\bX\s*=', line_s):
    nums = re.findall(r'[XYZ]\s*=\s*([+-]?\d+\.\d+[Ee][+-]?\d+)',
line_s)
    if len(nums) >= 3: pos = [float(n) for n in nums[:3]]
if vel is None and re.search(r'\bVX\s*=', line_s):
    nums = re.findall(r'[VXYZ]\s*=\s*([+-]?\d+\.\d+[Ee][+-]?\d+)',
line_s)
    if len(nums) >= 3: vel = [float(n) for n in nums[:3]]
if pos and vel: break
if pos and vel:
    return {"pos": pos, "vel": vel, "r": float(np.linalg.norm(pos))}
return None

```

Cálculo de métricas de error (eclipse_error)

```

def eclipse_error(pos_sol, pos_tierra, pos_luna, t_eclipse_pred=None,
referencia=None):
    # Separación angular Sol-Luna geocéntrica
    # Radios angulares y magnitud de fase
    # Error de posición de la Luna vs JPL
    # Error temporal vs hora NASA
    ...
    return {"sep_sol_luna_arcmin": ..., "fase_magnitud": ...,
            "es_total": ..., "d_luna_km": ..., "error_tiempo_min": ...}

```

Tabla comparativa (print_benchmark)

```

def print_benchmark(resultados):
    # Imprime tabla con columnas:
    # Modelo | d_Luna | Err_pos | Err_ang | Err_t | Total? | CPU
    ...

```

modelo2_ncuerpos.py

Layout del vector de estado (66 variables)

```

def ir(i): return 6*i      # posición del cuerpo i
def ip(i): return 6*i+3    # momento del cuerpo i

N_TRAS = 6*N              # 60 vars traslacionales

```

```
N_STATE = N_TRAS+6 # + 6 de rotación terrestre
ITH=60; IPH=61; IPS=62; IPTH=63; IPPH=64; IPPS=65
```

Derivadas: gravedad newtoniana N-cuerpos

```
for i in range(N):
    for j in range(i+1, N):
        dr = r[j] - r[i]
        d2 = np.dot(dr, dr)
        d3 = d2 * np.sqrt(d2)
        f = dr / d3
        ds[ip(i):ip(i)+3] += MASAS[i] * GM[j] * f
        ds[ip(j):ip(j)+3] -= MASAS[j] * GM[i] * f
```

Corrección J2 terrestre (pares Tierra–Luna y Tierra–Sol)

```
for ext in [I_LUN, I_SOL]:
    dr = r[ext] - r[I_TIE]
    d2 = np.dot(dr, dr)
    d = np.sqrt(d2)
    d5 = d2*d2*d
    u = np.dot(dr, up)
    C = 3.0*GM[ext]*dI/(2.0*d5)
    fJ2 = 5.0*u**2/d2 - 1.0
    g = C*(fJ2*dr - 2.0*u*up)
    ds[ip(ext):ip(ext)+3] -= g
    ds[ip(I_TIE):ip(I_TIE)+3] += g
```

Integrador Yoshida O(4) para N-cuerpos

```
_IQ = np.array([ir(i)+k for i in range(N) for k in range(3)] + [ITH, IPH, IPS])
_IP = np.array([ip(i)+k for i in range(N) for k in range(3)] + [IPTH, IPPH, IPPS])

def yoshida4(s, h):
    s = s.copy()
    for k in range(3):
        ds = derivadas(s)
        s[_IQ] += _C[k]*h*ds[_IQ]
        ds = derivadas(s)
        s[_IP] += _D[k]*h*ds[_IP]
    ds = derivadas(s)
```

```
s[_IQ] += _C[3]*h*ds[_IQ]
return s
```

modelo3_relativista.py

Velocidad de la luz en unidades del simulador

```
C_LUZ = 299792.458e3 / (AU / DAY) # m/s → AU/día
C2     = C_LUZ**2
```

Aceleración newtoniana pura (para usar en EIH)

```
def acels_newton(r):
    a = np.zeros((N, 3))
    for i in range(N):
        for j in range(i+1, N):
            dr = r[j] - r[i]
            d2 = np.dot(dr, dr)
            d3 = d2 * np.sqrt(d2)
            f = dr / d3
            a[i] += GM[j] * f
            a[j] -= GM[i] * f
    return a
```

Corrección post-newtoniana EIH

```
def correccion_eih(r, v, a_newton):
    # Para cada par (i, j):
    # Término escalar A = -vi2 - 2vj2 + 4(vi·vj) + 1.5(ĥ·vj)2 + 4Φi - Φj
    # Término vectorial B = 4(ĥ·vi) - 3(ĥ·vj)
    # da[i] += (GM[j] / c2r2) * (A·ĥ + B·dv)
    # + (3.5/c2) Σj (GM[j]/r) * ajNewton
    ...
```

Diagnóstico: magnitud relativa de la corrección EIH

```
a_n0 = acels_newton(r0)
da_eih0 = correccion_eih(r0, v0, a_n0)
ratio_tierra = np.linalg.norm(da_eih0[I_TIE]) / np.linalg.norm(a_n0[I_TIE])
```

```
ratio_luna = np.linalg.norm(da_eih0[I_LUN]) / np.linalg.norm(a_n0[I_LUN])
# Resultado típico: ~4.6e-8 (confirma que EIH es una corrección pequeña)
```

modelo4_besseliano.py

Rotación eclíptica → ecuatorial J2000

```
def ecliptic_to_equatorial(vec_ecl):
    x, y, z = vec_ecl
    cos_e = np.cos(EPSILON); sin_e = np.sin(EPSILON)
    return np.array([x, y*cos_e - z*sin_e, y*sin_e + z*cos_e])
```

Cálculo del ángulo sidéreo de Greenwich (GMST)

```
def gmst_angle(dt_utc):
    J2000 = datetime(2000, 1, 1, 12, 0, 0)
    d = (dt_utc - J2000).total_seconds() / 86400.0
    gmst_deg = ((18.697374558 + 24.06570982441908 * d) * 15.0) % 360.0
    return np.radians(gmst_deg)
```

Elementos besselianos del eclipse

```
def calcular_elementos_besselianos(rS_eq, rE_eq, rL_eq, utc_time):
    # Eje de la sombra k_hat = (R_L - R_S) / |R_L - R_S|
    # Base del plano fundamental (i_hat, j_hat, k_hat)
    # Coordenadas (x, y) de la sombra en el plano
    # Ángulo horario  $\mu$  = GMST - AR_sombra
    # Radios de penumbra l1 y umbra l2
    return {"x": x, "y": y, "d": d_rad, "mu": mu, "l1": l1, "l2": l2, ...}
```

Proyección sobre el elipsoide WGS84

```
def interseccion_wgs84(elementos):
    # Aproximación geocéntrica:
    # lat  $\approx \arcsin(\sin(d) \cdot \cos(y/a) + \cos(d) \cdot \sin(y/a))$ 
    # lon  $\approx -\mu + \arctan2(x, a \cdot \cos(lat))$ 
    return lat_deg, lon_deg
```

Flujo híbrido del Modelo 4

```
def run(h=0.05, verbose=True):  
    # 1. Integración física con Modelo 3 (N-cuerpos + EIH)  
    res3 = modelo3_relativista.run(h=h, verbose=False)  
    # 2. Conversión al marco ecuatorial en metros  
    rS_eq = ecliptic_to_equatorial(res3["_pos_sol"]) * AU  
    # 3. Elementos besselianos en el instante del eclipse simulado  
    elem = calcular_elementos_besselianos(rS_eq, rE_eq, rL_eq, t_ecl)  
    # 4. Proyección sobre WGS84  
    lat_deg, lon_deg = interseccion_wgs84(elem)
```